

Track B: Python for data science

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

- Track B: Python for data science2**
 - B1 Project setup 3
 - B1-L1 One pyproject.toml and a src layout 3
 - B1-L2 One virtual environment per project, and python -m pip 7
 - B1-L3 ruff and mypy in daily work 10
 - B2 Code design 15
 - B2-L1 Pure functions, side effects at the edges 15
 - B2-L2 Dataclasses as configuration objects 19
 - B2-L3 When a class earns its place 23
 - B3 Clean pandas 28
 - B3-L1 Dtypes in pandas 3, nullable and string 28
 - B3-L2 Vectorise instead of apply 31
 - B3-L3 Method chaining and Copy-on-Write 36
 - B4 Polars and DuckDB 41
 - B4-L1 Polars expressions and lazy frames 41
 - B4-L2 DuckDB, SQL over files and DataFrames 44
 - B4-L3 Choosing pandas, Polars, DuckDB or Spark 48
 - B5 Data validation 53
 - B5-L1 DataFrame schemas with pandera 53
 - B5-L2 pydantic models for config and input records 57
 - B5-L3 Validate at the boundaries, with severity 61
 - B6 Reproducibility 67
 - B6-L1 Seeds and random generators 67
 - B6-L2 Record the data, config and code behind every result 71
 - B6-L3 Pinned dependencies and lock files 74
 - Track B best-practice checklist 78
- Appendix: solutions and answer keys79**

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B1 Project setup

Roadmap: Prerequisite for all months. pyproject.toml, virtual environments, ruff, mypy and folder layout.

B1-L1 One pyproject.toml and a src layout

Concept

A data science project starts as a notebook and a CSV. By month three it has helper modules, tests, a model, and three people asking how to run it. The fix is structure from day one.

pyproject.toml is the one configuration file for a modern Python project. It holds:

- [project]: name, version, requires-python, and runtime dependencies.
- [dependency-groups] (PEP 735): named groups for development only, such as test or lint. They never ship with the package. Install one with `pip install --group test` (pip 25.1 and later).
- [tool.*] tables for tools: [tool.ruff], [tool.mypy], [tool.pytest.ini_options].

One file replaces setup.py, requirements.txt, setup.cfg, .flake8 and pytest.ini.

The src layout puts importable code under `src/`:

```
1 reach_model/
2 └─ pyproject.toml
3 └─ src/reach_model/      # importable package
4   └─ __init__.py
5   └─ features.py
6   └─ train.py
7 └─ tests/                # pytest, imports reach_model
8 └─ notebooks/           # exploration; imports reach_model, holds no
    logic
9 └─ data/                # small samples only; real data stays out of
    Git
```

With `src/`, Python can only import your package after you install it (`pip install -e .`). Tests then run against the installed package, exactly as a user or a job would. Without it, tests silently import whatever sits in the working directory, and "works on my machine" follows.

Notebooks explore and present. Functions that matter move into `src/`, where tests and type checks reach them. A notebook that defines the feature pipeline inline cannot be tested or reused by the training job.

Pin `requires-python` to the versions you test, for example `">=3.11"`, so pip refuses an interpreter you never tried.

Why it matters

A team trains a churn model from `train.ipynb`, which defines `build_features()` in a cell. The scoring job copies the function into `score.py`. A month later someone fixes a leakage bug in the notebook version only. Training and scoring now compute features differently, and live accuracy drops 9 points while offline metrics look fine.

With `build_features()` in `src/churn/features.py`, imported by both the notebook and the job, one fix reaches both, and one test protects it.

Worked example: weak code

1	churn/	
❶	├─ train.ipynb	# defines build_features() in cell 7
❷	├─ score.py	# copy of build_features(), edited later
4	├─ utils.py	
❸	├─ requirements.txt	# pandas, scikit-learn (no versions)
6	├─ setup.cfg	# flake8 settings
❹	└─ data.csv	# 2 GB client extract, committed

1. **Line 2.** Logic trapped in a notebook: untestable, and copied instead of imported.
2. **Line 3.** Two versions of one function drift apart.
3. **Line 5.** No pins: every install resolves different versions.
4. **Line 7.** Client data in Git: a size problem and a data-protection problem.

Worked example: strong code

1	churn/	
❶	├─ pyproject.toml	# project, dependency groups, ruff, mypy, pytest
3	├─ src/churn/	
4	├─ └─ __init__.py	
❷	├─ └─ features.py	# build_features(), used by training and scoring
6	├─ └─ train.py	
7	├─ └─ score.py	
8	├─ tests/	
❸	├─ └─ test_features.py	
10	├─ notebooks/	
11	├─ └─ 01_explore.ipynb	# imports churn.features

```
12 └─ data/
    4 └─ sample_200_rows.csv
```

1. **Line 2.** One file configures the project and every tool.
2. **Line 5.** One definition, imported everywhere it is needed.
3. **Line 9.** Tests import the installed package, as the jobs do.
4. **Line 13.** A small, anonymised sample for tests. Real data stays in governed storage.

Logic lives once, in an installable package. Notebooks, jobs and tests import it.

Recap card

- **What goes in `pyproject.toml`?** `[project]` with dependencies, `[dependency-groups]` for development tools, and `[tool.*]` settings for `ruff`, `mypy` and `pytest`.
- **Install a dependency group** `pip install --group dev (pip 25.1 and later)`.
- **Why the `src` layout?** Only the installed package is importable, so tests run against it exactly as jobs do.
- **Where does shared logic live?** In the package under `src/`, imported by notebooks, jobs and tests. Never copied.

Practice

Exercise 1. Multiple choice (level 1)

Where do you declare `pytest` and `ruff` so they install for developers but never ship with the package?

- a) `[project]` dependencies
- b) `[dependency-groups]`
- c) `[tool.pytest.ini_options]`
- d) A comment in `README.md`

Why?

- r1) Dependency groups (PEP 735) are named lists that are not part of the built distribution.
- r2) Dependency groups are only for test tools.

Exercise 2. Fill the gap (level 2)

Complete the `pyproject.toml` so runtime needs and developer tools are separate.

```
1 [project]
```

```

2 name = "churn"
3 requires-python = ">=3.11"
4 ____ (1) = ["pandas>=3.0,<3.1", "scikit-learn>=1.8,<1.10"]
5
6 [____ (2)]
7 dev = ["pytest>=9", "ruff>=0.16", "mypy>=2.3"]

```

Exercise 3. Write the function (level 3)

Write `unpinned(pyproject_text)`. Parse the TOML with the standard library and return, sorted, every entry in `[project].dependencies` that has no version constraint at all (no `=`, `<`, `>`, `~` or `!`). Return an empty list when there is no dependencies key.

Starter code:

```

1 import tomllib
2
3
4 def unpinned(pyproject_text: str) -> list[str]:
5     ...

```

Exercise 4. Multiple choice (level 4)

With a `src` layout, a colleague runs `pytest` straight after cloning and gets `ModuleNotFoundError: No module named 'churn'`. What is the right fix?

- a) Add `sys.path.insert(0, 'src')` at the top of every test file.
- b) Run `pip install -e .` in the project's virtual environment.
- c) Move the package out of `src/` so imports work from the root.
- d) Copy the package into `tests/`.

Why?

- r1) The `src` layout is designed so that only the installed package is importable, which makes tests use it like any user would.
- r2) `pytest` cannot find packages in folders.

Exercise 5. Architecture design (level 5)

Place each piece of a modelling project.

Options: `src/<package>/`, A notebook cell, `tests/`, Outside the repository.

1. `build_features()`, used by training and by the nightly scoring job
2. Plots comparing feature distributions for a one-off question
3. A check that `build_features()` gives the right output for 5 hand-made rows
4. The full client data extract

Sources

- PEP 735 defines [dependency-groups]; pip 25.1 (April 2025) added --group. (search-only: <https://peps.python.org/pep-0735/> and <https://ichard26.github.io/blog/2025/04/whats-new-in-pip-25.1/>)
- pip 26.2.1 lists --group in `pip install --help`; tomllib is in the standard library from Python 3.11. (verified: Checked in this repository's build environment.)

B1-L2 One virtual environment per project, and python -m pip

Concept

A **virtual environment** is a folder with its own Python interpreter link and its own `site-packages`. Packages you install there do not touch the system Python or any other project.

```
1 python -m venv .venv
2 source .venv/bin/activate      # Windows: .venv\Scripts\activate
3 python -m pip install --upgrade pip
4 python -m pip install -e . --group dev
```

Rules that prevent most "it worked yesterday" problems:

- **One environment per project.** Two projects that share an environment share their version conflicts. Upgrading pandas for one breaks the other.
- **Never install into the system Python.** Operating systems and other tools depend on it.
- **Use** `python -m pip`, **not bare** `pip`. Bare `pip` runs whichever `pip` comes first on your PATH, which may belong to a different interpreter. `python -m pip` always installs into the interpreter you are about to run.
- **Keep** `.venv/` **out of Git.** It is large, machine-specific and rebuildable. Commit `pyproject.toml` and a lock file instead (lesson B6-L3).

In code, `sys.prefix != sys.base_prefix` is True inside a virtual environment. A script that must not run against the system Python can check it and stop.

Sometimes one project needs **two** environments, because two dependencies cannot live together. A real case from this course's own build: PySpark 4.0.4 fails to import its pandas integration with pandas 3.x, so the Spark test harness runs in its own environment pinned to pandas 2.x, while the rest uses pandas 3. Separate environments, each fully pinned, beat one environment held together by guesswork.

Tools such as `uv` create and sync environments much faster, and `conda` manages non-Python libraries. The principles are the same.

Why it matters

An analyst runs `pip install statsmodels` in a terminal, then `python analysis.py` in the same terminal, and gets `ModuleNotFoundError`. `pip` belonged to one Python, and `python` to another. They try `sudo pip install`, which upgrades NumPy in the system Python and breaks a command-line tool the operating system uses. Two hours are gone.

`python -m venv .venv`, activation, and `python -m pip install` make the interpreter that installs and the interpreter that runs the same one.

Worked example: weak code

```
❶ $ pip install pandas scikit-learn
❷ $ sudo pip install --upgrade numpy
3 $ python train.py
❸ ModuleNotFoundError: No module named 'sklearn'
❹ $ git add .venv/ data/ train.py
```

1. **Line 1.** Bare pip, no environment: installs into whichever Python owns this pip.
2. **Line 2.** Changes the system Python that the operating system relies on.
3. **Line 4.** The python that runs is not the one pip installed into.
4. **Line 5.** Commits a machine-specific environment folder and data.

Worked example: strong code

```
❶ $ python -m venv .venv
2 $ source .venv/bin/activate
3 (.venv) $ python -m pip install --upgrade pip
❷ (.venv) $ python -m pip install -e . --group dev
5 (.venv) $ python -c "import sys; print(sys.prefix != sys.base_prefix)"
❸ True
7 (.venv) $ python train.py
```

1. **Line 1.** A project-local environment, listed in `.gitignore`.
2. **Line 4.** The same interpreter installs and runs. Dependencies come from `pyproject.toml`.
3. **Line 6.** Proof that the active interpreter is the virtual environment.

One environment per project, created from `pyproject.toml`, with `python -m pip` so installing and running use the same interpreter.

Recap card

- **Create and use a project environment** `python -m venv .venv`, activate it, then `python -m pip install -e . --group dev`.

- **Why python -m pip?** It installs into the interpreter that python runs. Bare pip may belong to another Python.
- **Detect a virtual environment in code** `sys.prefix != sys.base_prefix`.
- **Two dependencies that cannot coexist** Two pinned environments and two dependency groups, each tested in CI.

Practice

Exercise 1. Multiple choice (level 1)

Why prefer `python -m pip install x` over `pip install x`?

- a) It is faster.
- b) It installs into the interpreter that `python` runs, whatever is first on `PATH`.
- c) It installs packages globally for all users.
- d) It skips dependency resolution.

Why?

- r1) `python -m pip` runs pip as a module of that exact interpreter.
- r2) Bare pip is deprecated.

Exercise 2. Predict the output (level 2)

You run this with `.venv/bin/python`, a virtual environment created by `python -m venv .venv`. What does it print?

```
1 import sys
2
3 print(sys.prefix != sys.base_prefix)
```

Exercise 3. Fill the gap (level 2)

Create the environment, then install the project in editable mode with its dev group.

```
1 python -m ____ (1) .venv
2 source .venv/bin/activate
3 python -m pip install ____ (2) . --group dev
```

Exercise 4. Write the function (level 3)

Write `require_virtualenv()`. It raises `RuntimeError` with the message "activate the project's virtual environment first" when the running interpreter is not inside a virtual environment, and returns `None` otherwise. Read `sys.prefix` and `sys.base_prefix` when the function is called, not at import time.

Starter code:

```
1 import sys
2
3
4 def require_virtualenv() -> None:
5     ...
```

Exercise 5. Architecture design (level 5)

Your project needs pandas 3.0 for analysis, and a Spark test harness on PySpark 4.0.4, which fails to import its pandas integration with pandas 3.x. Choose the set-up.

Options: Two environments: dev (pandas 3) and spark (pandas 2), each pinned, One environment with the latest of everything, One environment, everything downgraded to pandas 2.

1. Environments
2. Where the difference is recorded
3. CI

Sources

- `sys.prefix` differs from `sys.base_prefix` inside a `venv`. (verified: Python `venv` documentation; checked in this repository's virtual environments.)
- PySpark 4.0.4 fails to import `assertDataFrameEqual` with pandas 3.x. (verified: Reproduced in this repository (VERSIONS.md).)

B1-L3 ruff and mypy in daily work

Concept

Two tools catch most mistakes before a test runs.

ruff lints and formats. On a ten-line sample it reported three real problems:

```
1 m.py:1:1: I001 [*] Import block is un-sorted or un-formatted
2 m.py:1:8: F401 [*] `os` imported but unused
3 m.py:9:30: B905 `zip()` without an explicit `strict=` parameter
```

B905 matters for data work: `zip(flags, weights)` silently stops at the shorter list, so a missing weight drops a respondent without an error. `zip(..., strict=True)` raises instead.

mypy checks types. On the same file, strict mode found:

```
1 m.py:14: error: Unsupported operand types for * ("None" and "int")
  [operator]
2 m.py:17: error: Function is missing a type annotation [no-untyped-def]
```

Line 14 multiplies a `float | None` by 100. When the total weight is zero, the function returns `None` and the report crashes. `mypy` found the zero-weight bug without any test data.

Daily habits:

- Configure both once in `pyproject.toml` (`[tool.ruff]` with the rule sets you want, `[tool.mypy]` with `strict = true`).
- Run `ruff check --fix .` and `ruff format .` before each commit. `--fix` applies only safe fixes.
- Run `mypy` on `src/` and `tests/`. Fix errors by handling the case, not by adding `# type: ignore`.
- Run the same commands in CI (lesson A10-L3), so the rules are not optional.

Libraries without type information show `import-untyped` or `missing-stub` notes. Install the stub package if one exists, or add a targeted `[[tool.mypy.overrides]]` with `ignore_missing_imports = true` for that module only.

Why it matters

A reporting script formats a weighted share as `f"{share * 100:.1f}%"`. For eleven months every subgroup has respondents. Then a new region with no fieldwork yet appears in the sample frame, its total weight is zero, `weighted_share` returns `None`, and the monthly client pack fails to build on deadline day.

`mypy` flags `None * 100` the first time it runs on the file. The fix is two lines, written months before the failure.

Worked example: weak code

```
1 import os
2 import pandas as pd
3
4
5 def weighted_share(flags, weights):
6     total = sum(weights)
7     if total == 0:
8         return None
9
10    return sum(w for f, w in zip(flags, weights) if f == 1) / total
11
12 def report(flags, weights):
13     return f"{weighted_share(flags, weights) * 100:.1f}%"
```

1. **Line 1.** F401: unused import.

2. **Line 5.** No hints: mypy cannot see that the function can return None.
3. **Line 9.** B905: zip silently truncates to the shorter list.
4. **Line 13.** Crashes with TypeError when the share is None.

Worked example: strong code

```

1 def weighted_share(flags: list[int], weights: list[float]) -> float |
  None:
2     """Weighted share of flagged rows, or None when the total weight is
   0."""
3     total = sum(weights)
4     if total == 0:
5         return None
6     return sum(w for f, w in zip(flags, weights, strict=True) if f ==
   1) / total
7
8
9 def report(flags: list[int], weights: list[float]) -> str:
10     """Share as a percentage string, or 'n/a' when undefined."""
11     share = weighted_share(flags, weights)
12
13     if share is None:
14         return "n/a"
15     return f"{share * 100:.1f}%"

```

1. **Line 1.** The return type admits None, so mypy makes every caller handle it.
2. **Line 6.** Unequal lengths raise ValueError instead of dropping rows.
3. **Line 12.** The case mypy flagged, handled on purpose.

ruff and mypy found an unused import, a silent truncation and a crash on zero weight, before anyone wrote a test.

Recap card

- **Why does ruff flag zip() without strict= (B905)?** zip silently truncates to the shorter input. strict=True raises ValueError instead.
- **mypy says None * int is unsupported. The fix?** Handle the None case in code. Never # type: ignore it away.
- **Daily commands before a commit** ruff check --fix ., ruff format ., mypy.
- **Missing stubs for one library** Install types-<lib> if it exists, otherwise a per-module mypy override. Never global.

Practice

Exercise 1. Predict the output (level 1)

What does this print?

```

1 flags = [1, 0, 1]
2 weights = [2.0, 1.0]

```

```
3 print(list(zip(flags, weights)))
```

Exercise 2. Multiple choice (level 2)

mypy reports `Unsupported operand types for * ("None" and "int")` on `share * 100`. What is the right fix?

- a) Add `# type: ignore` to the line.
- b) Handle the `None` case before the multiplication.
- c) Change the return type to `float` and keep returning `None`.
- d) Turn off strict mode.

Why?

- r1) mypy found a real path where the value is `None`. Handling it removes the crash.
- r2) mypy errors are style issues.

Exercise 3. Fill the gap (level 2)

Complete the pre-commit routine: apply safe lint fixes, format, then type-check.

```
1 ruff check ____ (1) .
2 ruff ____ (2) .
3 ____ (3)
```

Exercise 4. Refactor (level 4)

Make this pass `ruff` (no B905) and `mypy --strict`: add type hints, keep the `None` result for zero weight, make unequal lengths raise, and make `report` return `"n/a"` instead of crashing.

Starter code:

```
1 def weighted_share(flags, weights):
2     total = sum(weights)
3     if total == 0:
4         return None
5     return sum(w for f, w in zip(flags, weights) if f == 1) / total
6
7
8 def report(flags, weights):
9     return f"{weighted_share(flags, weights) * 100:.1f}%"
```

Exercise 5. Architecture design (level 5)

mypy reports `Skipping analyzing "shapely": module is installed, but missing library stubs or py.typed marker`. Choose the response for each case.

Options: Install the stubs in the dev group, Add `[[tool.mypy.overrides]]` with `ignore_missing_imports` for that module, Set `ignore_missing_imports = true` globally.

1. A stub package, `types-shapely`, exists on PyPI
2. No stubs exist for an internal library
3. A colleague proposes silencing all such messages for every library

Sources

- ruff reports `I001`, `F401` and `B905` and mypy `--strict` reports `[operator]` and `[no-untyped-def]` on the sample shown. (verified: ruff 0.16.9 and mypy 2.3.1 run in this repository.)
- `zip(strict=True)` raises `ValueError` on unequal lengths (Python 3.10+). (verified: Python standard library; run in Python 3.11.)

B2 Code design

Roadmap: Prerequisite for all months. Pure functions, dataclasses, when to use classes, configuration objects.

B2-L1 Pure functions, side effects at the edges

Concept

A **pure function** returns a value that depends only on its arguments, and changes nothing outside itself: no file writes, no global variables, no mutation of the inputs, no printing, no clock or random calls it does not receive.

```
1 def weighted_mean(values: list[float], weights: list[float]) -> float:
2     return sum(v * w for v, w in zip(values, weights, strict=True)) /
    sum(weights)
```

Pure functions are easy to test (inputs in, answer out), safe to reuse, and safe to run twice. Most analysis logic can be pure: cleaning, feature building, statistics, scoring.

Some work is not pure by nature: reading files, calling APIs, writing tables, logging, reading the clock. Push that work to the **edges** of the program, into a thin layer that gathers inputs, calls the pure core, and writes results.

```
1 read (impure) -> transform, compute (pure) -> write (impure)
```

Three side effects hide easily in data code:

- **Mutating an argument.** `df["x"] = ...` or `rows.append(...)` inside a function changes the caller's object. The caller did not ask for that. Return a new object instead: `df.assign(x=...)`, or build a new list.
- **Reading globals.** A function that reads a module-level `THRESHOLD` or `TODAY` gives different answers after someone changes it. Pass it in.
- **Hidden time and randomness.** `datetime.now()` or `np.random.rand()` inside a function makes its output untestable. Pass a date, or a `numpy.random.Generator`.

In pandas 3, Copy-on-Write is always on, so many operations already return new objects. Assigning a column with `df[col] = ...` still changes `df` itself, which is exactly the mutation to keep out of shared functions.

Why it matters

A feature function adds `age_group` to the DataFrame it receives, in place. The training script calls it, then passes the same DataFrame to a stratified split on `age_group`. Later, someone reorders the calls so the split runs first. The split fails with a `KeyError` on one laptop and works on another, depending on which notebook cells ran before.

A pure `add_age_group(df) -> DataFrame` that returns a new frame gives the same result in every order, and its test needs three rows.

Worked example: weak code

```
1 import datetime
2
3 THRESHOLD = 0.5
4
5
6 def score_respondents(df):
7     df["age"] = datetime.date.today().year - df["birth_year"]
8     df["high_reach"] = df["reach"] > THRESHOLD
9     df.to_csv("scored.csv", index=False)
10    print("scored", len(df))
11    return df
```

1. **Line 7.** Mutates the caller's frame and reads the clock: the answer changes on 1 January.
2. **Line 8.** Reads a module global: change `THRESHOLD` anywhere and every result changes.
3. **Line 9.** Writes a file from inside the computation.
4. **Line 10.** Output in the middle of logic, not in the program's edge.

Worked example: strong code

```
1 import pandas as pd
2
3
4 def score_respondents(df: pd.DataFrame, as_of_year: int, threshold:
5     float) -> pd.DataFrame:
6     """Add age and high_reach. Returns a new frame; the input is
7     unchanged."""
8     return df.assign(
9         age=as_of_year - df["birth_year"],
10        high_reach=df["reach"] > threshold,
11    )
12
13 def main(input_path: str, output_path: str, as_of_year: int, threshold:
14     float) -> None:
15     """The edge: read, call the pure core, write."""
```

```

14     scored = score_respondents(pd.read_csv(input_path), as_of_year,
    threshold)
15     scored.to_csv(output_path, index=False)

```

1. **Line 4.** Everything the result depends on is a parameter.
2. **Line 6.** assign returns a new frame. The caller's data is not touched.
3. **Line 12.** Reading, writing and choosing the year happen here, at the edge.

The pure core is tested with three rows and a fixed year. The edge is a few lines that are easy to read.

Recap card

- **What makes a function pure?** Its result depends only on its arguments, and it changes nothing outside itself.
- **Three hidden side effects in data code** Mutating arguments, reading globals, reading the clock or randomness inside the function.
- **Return a new DataFrame with an extra column** df.assign(col=...). df["col"] = ... changes the caller's frame.
- **Program shape** Read at the edge, compute in a pure core, write at the edge.

Practice

Exercise 1. Multiple choice (level 1)

Which function is pure?

- a) `def add(rows, x): rows.append(x); return rows`
- b) `def share(n, total): return n / total if total else None`
- c) `def stamp(df): return df.assign(ts=pd.Timestamp.now())`
- d) `def save(df, path): df.to_csv(path)`

Why?

- r1) Its result depends only on its arguments, and it changes nothing outside itself.
- r2) It is the shortest.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3?

```

1 import pandas as pd
2
3
4 def add_flag(df):
5     df["flag"] = df["x"] > 1

```

```

6     return df
7
8
9 original = pd.DataFrame({"x": [1, 2]})
10 result = add_flag(original)
11 print(list(original.columns))

```

Exercise 3. Refactor (level 3)

Make `add_age` pure: take the reference year as a parameter, return a new DataFrame, and leave the input unchanged. Keep the name `add_age` and the new column `age`.

Starter code:

```

1 import datetime
2
3
4 def add_age(df):
5     df["age"] = datetime.date.today().year - df["birth_year"]
6     return df

```

Exercise 4. Spot the bug (level 4)

The second call returns a list containing the first call's flags. Tap the line, then fix the function without changing its behaviour otherwise.

```

1 def flag_low_bases(bases: dict[str, float], minimum: float, flags:
    list[str] = []) -> list[str]:
2     """Names of subgroups whose base is below minimum."""
3     for name, base in bases.items():
4         if base < minimum:
5             flags.append(name)
6     return flags

```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

Sort each step of a weekly scoring script into the pure core or the edge.

Options: Edge, Pure core.

1. Read the respondent extract from storage
2. Compute features from the extract and a reference date
3. Score respondents with a model object passed in
4. Write scores to the output table and log the row count

Sources

- In pandas 3 Copy-on-Write is always on, but assigning a column changes the DataFrame it is called on. (verified: Run on pandas 3.0.6 and 3.0.2 in this repository (predict-output exercise).)

B2-L2 Dataclasses as configuration objects

Concept

Analysis code needs settings: a minimum base size, a weight column, a list of waves. Passing them around as a loose `dict` has three costs. A typo in a key (`"min_bse"`) fails late or not at all. Nothing states which keys exist. Any function can change a value half way through a run.

A **dataclass** (Python 3.7 and later) turns the settings into a named, typed object.

```
1 from dataclasses import dataclass, field
2
3
4 @dataclass(frozen=True)
5 class ReportConfig:
6     weight_col: str
7     min_base: int = 50
8     waves: tuple[str, ...] = ()
9
10     def __post_init__(self) -> None:
11         if self.min_base <= 0:
12             raise ValueError(f"min_base must be positive, got {self.min_base}")
```

What you get:

- **Named fields.** `cfg.min_bse` raises `AttributeError` at once. Your editor and mypy see the fields.
- `frozen=True`. Assigning a field raises `dataclasses.FrozenInstanceError`. Settings stay fixed for the run. Use `dataclasses.replace(cfg, min_base=30)` to build a changed copy.
- `__post_init__`. Runs after the generated `__init__`. Put checks here so a bad config fails at load time, not in hour three.

Two traps:

- **Type hints are not checked at runtime.** `ReportConfig(weight_col="w", min_base="30")` builds without error. Convert or check types in `__post_init__`, or use `pydantic` (lesson B5-L2) when input comes from files.

- **Mutable defaults.** `waves: list[str] = []` raises `ValueError` when the class is defined. Use `field(default_factory=list)`, or an immutable tuple.

Load the config once at the edge, from YAML, JSON or arguments, and pass the object to the pure core.

Why it matters

A weighting script reads `config["min_base"]` from a dict built in a notebook cell. An analyst edits the cell to `config["minbase"] = 30` for a test and forgets to revert. The script keeps using the old default of 50, so 14 subgroups the client asked to see are suppressed in the published tables. Nobody notices for two waves.

With a frozen dataclass, `ReportConfig(minbase=30)` raises `TypeError: unexpected keyword argument` at the first run.

Worked example: weak code

```

1 CONFIG = {"weight_col": "weight", "min_base": 50, "waves": []}
2
3
4 def suppress_small(counts, config=CONFIG):
5     config["min_base"] = config.get("minbase", 50)
6     return {k: v for k, v in counts.items() if v >= config["min_base"]}

```

1. **Line 1.** A module-level dict: no fixed keys, no types, shared and mutable.
2. **Line 4.** A mutable default that every call shares.
3. **Line 5.** A typo in the key silently falls back to 50, and the write changes the shared dict.

Worked example: strong code

```

1 from dataclasses import dataclass
2
3
4 @dataclass(frozen=True)
5 class ReportConfig:
6     """Settings for one reporting run."""
7
8     weight_col: str
9     min_base: int = 50
10
11     def __post_init__(self) -> None:
12         if not isinstance(self.min_base, int) or self.min_base <= 0:
13             raise ValueError(f"min_base must be a positive int, got {self.min_base!r}")
14
15

```

```

3 def suppress_small(counts: dict[str, int], config: ReportConfig) ->
  dict[str, int]:
17     """Keep subgroups whose base reaches config.min_base."""
18     return {k: v for k, v in counts.items() if v >= config.min_base}

```

1. **Line 4.** Fixed fields, and no assignment after construction.
2. **Line 12.** Validation at load time. Hints alone are not checked at runtime.
3. **Line 16.** The function receives the config and reads it. It never changes it.

A frozen dataclass names every setting, rejects unknown ones, and checks values once, when you build it.

Recap card

- **Why a frozen dataclass for settings?** Named, typed fields, unknown keys rejected, and no assignment after construction.
- **Change one setting on a frozen config** `dataclasses.replace(cfg, min_base=30)` returns a new object.
- **Are dataclass type hints checked at runtime?** No. Validate or convert in **post_init**, or use `pydantic` for external input.
- **Empty list default on a dataclass field** `field(default_factory=list)`. A plain `[]` raises `ValueError` at class definition.

Practice

Exercise 1. Multiple choice (level 1)

`cfg` is an instance of a `@dataclass(frozen=True)` class with a field `min_base`. What does `cfg.min_base = 30` do?

- a) Sets `min_base` to 30.
- b) Raises `dataclasses.FrozenInstanceError`.
- c) Creates a copy with `min_base` 30.
- d) Does nothing, silently.

Why?

- r1) `frozen=True` makes the generated **setattr** raise on every field assignment.
- r2) Integer fields are read-only in every dataclass.

Exercise 2. Predict the output (level 2)

What does this print?

```

1 from dataclasses import dataclass
2
3

```

```

4 @dataclass
5 class Settings:
6     min_base: int = 50
7
8
9 print(Settings("30"))

```

Exercise 3. Fill the gap (level 2)

Give `waves` an empty list as its default, safely.

```

1 from dataclasses import dataclass, field
2
3
4 @dataclass
5 class Run:
6     waves: list[str] = field(____(1)=list)

```

Exercise 4. Write the function (level 3)

Complete `ReportConfig` and write `load_config(raw)`.

- `ReportConfig` is a frozen dataclass with `weight_col: str` and `min_base: int = 50`.
- In `__post_init__`, raise `ValueError` if `min_base` is not an `int` or is not positive.
- `load_config(raw: dict[str, object]) -> ReportConfig` builds the config from a dict. An unknown key must raise `TypeError`, a missing `weight_col` must raise `TypeError`.

Starter code:

```

1 from dataclasses import dataclass
2
3
4 class ReportConfig:
5     ...
6
7
8 def load_config(raw):
9     ...

```

Exercise 5. Architecture design (level 4)

A weighting pipeline reads its settings from `weighting.yaml`. Choose how the settings flow through the code.

Options: A frozen dataclass with typed fields, The dict returned by `yaml.safe_load`, Module-level constants set from the file at import.

1. Shape of the settings in code
2. When values are checked

3. How functions get the settings

Sources

- Assigning a field of a frozen dataclass raises `dataclasses.FrozenInstanceError`, a subclass of `AttributeError`. (verified: Python dataclasses documentation; run on Python 3.11 and 3.14 in this repository.)
- A list default on a dataclass field raises `ValueError` at class definition. (verified: Python dataclasses documentation; run on Python 3.11 in this repository.)
- Dataclasses do not check or convert field types at runtime. (verified: Python dataclasses documentation; predict-output exercise run in this repository.)

B2-L3 When a class earns its place

Concept

Python does not require classes. Many data science classes are functions in disguise: an `__init__` that stores arguments, and one method that uses them.

```
1 class Normaliser:
2     def __init__(self, col):
3         self.col = col
4
5     def run(self, df):
6         return df.assign(**{self.col: df[self.col] / df[self.col].sum()})
```

Every caller writes `Normaliser("spend").run(df)`. A function, `normalise(df, col)`, says the same thing in one call, with nothing to construct.

A class earns its place when at least one of these holds:

- **State that outlives one call.** A weighting model learns targets in `fit` and applies them in `transform`. scikit-learn's `StandardScaler` stores `mean_` and `scale_` between the two calls.
- **An invariant to protect.** A `Weights` object guarantees values are positive and sum to the sample size. Code that builds one through its constructor cannot break that rule.
- **Several operations on the same data.** A `SurveyDesign` with `mean()`, `ci()` and `subgroup()` that all need weights, strata and clusters.

- **A shared interface.** Several classes with the same method, such as `load()`, so the caller swaps one for another.

Settings with no behaviour belong in a dataclass (lesson B2-L2). A single step belongs in a function (lesson B2-L1).

Test a design by asking: if you delete the class and keep its methods as functions, what breaks? If nothing, keep the functions.

Why it matters

A team wraps each cleaning step in a class: `DropDuplicates`, `FixDates`, `RenameColumns`, each with `__init__` and `run`. A new analyst adds a `self.seen_ids` set to `DropDuplicates` to speed up a second call. The pipeline object is reused across two tracker waves, so wave 2 silently drops every respondent who also took part in wave 1: 38% of the sample.

Pure functions have no `self` to keep state state between calls.

Worked example: weak code

```

1 class AgeBander:
2     def __init__(self, bins, labels):
3         self.bins = bins
4         self.labels = labels
5
6     def run(self, df):
7         return df.assign(age_band=pd.cut(df["age"], self.bins,
8             labels=self.labels))
9
10 banded = AgeBander([0, 34, 54, 120], ["18-34", "35-54", "55+"]).run(df)

```

1. **Line 1.** No state outlives a call, no invariant, one operation: a function in disguise.
2. **Line 6.** A single method named `run` is the usual sign.
3. **Line 10.** Every caller builds an object only to call one method.

Worked example: strong code

```

1 import pandas as pd
2
3
4 def add_age_band(df: pd.DataFrame, bins: list[int], labels: list[str]) ->
5     pd.DataFrame:
6     """Return df with age_band cut from age."""
7     return df.assign(age_band=pd.cut(df["age"], bins, labels=labels))
8
9 class RakingWeights:

```

```

10     """Learn raking targets on one wave, apply them to later waves."""
11
12     def __init__(self, targets: dict[str, dict[str, float]]) -> None:
13         self.targets = targets
14         self.factors_: dict[str, float] | None = None
15
16     def fit(self, df: pd.DataFrame) -> "RakingWeights":
17         ...
18         return self
19
20     def transform(self, df: pd.DataFrame) -> pd.Series:
21         if self.factors_ is None:
22             raise RuntimeError("call fit before transform")
23         ...

```

1. **Line 4.** One step, no state: a function.
2. **Line 14.** State that outlives one call, learned in fit and used in transform.
3. **Line 22.** The class protects an order of operations a function could not.

Functions by default. A class when state lives across calls, an invariant needs guarding, or several methods share the same data.

Recap card

- **A class with init and one run method** A function in disguise. Replace it with a function.
- **Four reasons a class earns its place** State across calls, an invariant to protect, several operations on the same data, a shared interface.
- **The deletion test** Turn the methods into functions. If nothing breaks, keep the functions.
- **Settings with no behaviour** A frozen dataclass.

Practice

Exercise 1. Multiple choice (level 1)

Which of these most needs a class?

- a) Rename columns using a mapping.
- b) Learn category frequencies on training data and encode test data with them.
- c) Convert a date column to UTC.
- d) Hold three settings with no behaviour.

Why?

- r1) State learned in one call must be reused in a later call.
- r2) It has the most lines of code.

Exercise 2. Predict the output (level 2)

What does this print?

```
1 class Deduper:
2     def __init__(self) -> None:
3         self.seen: set[int] = set()
4
5     def run(self, ids: list[int]) -> list[int]:
6         out = [i for i in ids if i not in self.seen]
7         self.seen.update(ids)
8         return out
9
10
11 d = Deduper()
12 d.run([1, 2, 3])
13 print(d.run([2, 3, 4]))
```

Exercise 3. Refactor (level 3)

Replace the class with a function `add_share(df, col)` that returns a new DataFrame with a column `share` equal to `df[col] / df[col].sum()`. If the column sums to zero, set `share` to `0.0` for every row.

Starter code:

```
1 class ShareAdder:
2     def __init__(self, col):
3         self.col = col
4
5     def run(self, df):
6         return df.assign(share=df[self.col] / df[self.col].sum())
```

Exercise 4. Spot the bug (level 4)

The same `Scaler` object is fitted on wave 1 and used for waves 2 and 3. Wave 3 results match a scaler fitted on wave 2 only. Tap the line that causes it.

```
1 class Scaler:
2     """Standardise with the mean and sd learned in fit."""
3
4     def fit(self, values: list[float]) -> "Scaler":
5         n = len(values)
6         self.mean_ = sum(values) / n
7         self.sd_ = (sum((v - self.mean_) ** 2 for v in values) / n) ** 0.5
8         return self
9
10    def transform(self, values: list[float]) -> list[float]:
11        self.fit(values)
12        return [(v - self.mean_) / self.sd_ for v in values]
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

Choose a form for each piece of a survey analysis package.

Options: Function, Frozen dataclass, Class with methods.

1. Recode a Likert column to numbers using a mapping
2. Confidence level, minimum base and weight column
3. Weights, strata and clusters used by mean, CI and subgroup methods

Sources

- scikit-learn's StandardScaler stores mean_ and scale_ after fit. (verified: scikit-learn 1.8 API reference; checked in this repository's dev environment.)

B3 Clean pandas

Roadmap: Prerequisite for all months. Dtypes, vectorisation, method chaining and Copy-on-Write in pandas 3.

B3-L1 Dtypes in pandas 3, nullable and string

Concept

Every pandas column has a **dtype**. The wrong dtype gives wrong answers without an error: IDs that lose leading zeros, counts that turn into floats, text that sorts as numbers.

Three facts about pandas 3.0:

- **Text columns default to the `str` dtype.** `pd.Series(["a", None]).dtype` prints `str`, not `object`. Missing values in it are `NaN`. Only strings are allowed, so a stray integer no longer hides in a text column.
- **A missing value turns an integer column into `float64`,** unless you ask for a nullable dtype. `pd.Series([1, None])` is `float64`. `pd.Series([1, None], dtype="Int64")` keeps integers and shows the gap as `<NA>`.
- **Setting a value that does not fit raises.** Assigning `1.5` into an `int64` Series raises `TypeError` instead of silently upcasting the column. (Assigning `NaN` still upcasts to `float64`.)

The capital letter matters. `int64` is NumPy's integer with no missing values. `Int64`, `Float64` and `boolean` are pandas **nullable** dtypes that hold `pd.NA`.

Set dtypes where data enters, not later:

```
1 df = pd.read_csv(  
2     path,  
3     dtype={"respondent_id": "str", "postcode": "str", "household_size":  
4         "Int64"},  
5 )
```

Without the `dtype` map, `read_csv` reads a postcode such as `01234` as the number `1234.0` once any row is blank. With it, `"01234"` stays a string.

Check the result with `df.dtypes` right after loading. `df.convert_dtypes()` switches every column to the nullable equivalents in one call, useful for exploration. An explicit map is better in pipelines, because it fails loudly when the source changes.

Why it matters

A panel file stores respondent IDs such as 00731. One wave has a blank ID row, so `read_csv` infers `float64` and the ID becomes 731.0. The join to the previous wave, where IDs were read as text, matches nothing. The longitudinal report shows 0% retention for a panel that kept 82% of its members.

`dtype={"respondent_id": "str"}` at load time keeps 00731 as written in every wave.

Worked example: weak code

```
❶ df = pd.read_csv("panel_w2.csv")
❷ df["respondent_id"] = df["respondent_id"].astype(int).astype(str)
❸ df["household_size"] = df["household_size"].fillna(0).astype(int)
```

1. **Line 1.** Dtypes inferred: a blank row turns IDs and counts into float64.
2. **Line 2.** Leading zeros were lost at load time. This cannot bring them back.
3. **Line 3.** Replaces 'unknown' with 0, a real value that biases every mean.

Worked example: strong code

```
1 import pandas as pd
2
3 PANEL_DTYPES: dict[str, str] = {
❶     "respondent_id": "str",
❷     "household_size": "Int64",
6 }
7
8
9 def read_panel(path: str) -> pd.DataFrame:
10     """Read one panel wave with explicit dtypes."""
❸     return pd.read_csv(path, dtype=PANEL_DTYPES)
```

1. **Line 4.** IDs are labels, not numbers: 00731 stays 00731.
2. **Line 5.** Nullable integer: a gap stays <NA>, the rest stay integers.
3. **Line 11.** Dtypes fixed where data enters. In a pipeline, load the map from config.

Decide dtypes at load time. IDs and codes as str, counts with gaps as Int64.

Recap card

- **Default text dtype in pandas 3.0** str, with NaN for missing values. Only strings are allowed.
- **int64 vs Int64** int64 is NumPy, no missing values. Int64 is the pandas nullable integer that holds pd.NA.
- **Keep leading zeros in IDs** Read the column with dtype str in `read_csv`.

- **Assign 1.5 into an int64 Series on pandas 3** Raises `TypeError` instead of silently upcasting.
- **Where to set dtypes** At load time, from a dtype map in config. Check `df.dtypes` right after.

Practice

Exercise 1. Multiple choice (level 1)

On pandas 3.0, what is the dtype of `pd.Series([4, None, 2])`?

- a) `int64`
- b) `Int64`
- c) `float64`
- d) `object`

Why?

- r1) NumPy `int64` cannot hold a missing value, so pandas stores `NaN` in a float column.
- r2) pandas 3 stores every number as float.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3.0?

```
1 import io
2
3 import pandas as pd
4
5 csv = "id,postcode\n1,01234\n2,\n"
6 df = pd.read_csv(io.StringIO(csv))
7 print(df["postcode"].iloc[0])
```

Exercise 3. Fill the gap (level 2)

Keep IDs as text and let the count column hold missing values as integers.

```
1 df = pd.read_csv(path, dtype={"respondent_id": "____(1)",
    "household_size": "____(2)"})
```

Exercise 4. Write the function (level 3)

Write `read_panel(text: str) -> pd.DataFrame`. It reads CSV text with columns `respondent_id` and `household_size`. Keep `respondent_id` as text with leading zeros, and `household_size` as a nullable integer where blanks become `<NA>`. Take the dtype map from a module constant `PANEL_DTYPES`.

Starter code:

```

1 import io
2
3 import pandas as pd
4
5
6 def read_panel(text):
7     return pd.read_csv(io.StringIO(text))

```

Exercise 5. Spot the bug (level 4)

The mean household size reported for wave 3 dropped from 2.6 to 2.1. Nothing changed in the sample. Tap the line that caused it.

```

1 import pandas as pd
2
3
4 def mean_household(df: pd.DataFrame) -> float:
5     """Mean household size among respondents who answered."""
6     sizes = df["household_size"].astype("Int64")
7     sizes = sizes.fillna(0)
8     return float(sizes.mean())

```

Write down the line number of the bug. Then write the fix.

Sources

- In pandas 3.0 text columns default to the str dtype, with NaN as the missing value. (verified: pandas 3.0 release notes; run on pandas 3.0.6 and 3.0.2 in this repository.)
- In pandas 3.0 setting an incompatible value such as 1.5 into an int64 Series raises TypeError. (verified: pandas 3.0 release notes (setitem upcasting removed); run on pandas 3.0.6 and 3.0.2 in this repository.)
- read_csv infers float64 for an integer-like column with a blank value. (verified: Run on pandas 3.0.6 and 3.0.2 in this repository.)

B3-L2 Vectorise instead of apply

Concept

pandas stores each column as an array. A **vectorised** operation, such as `df["a"] * df["b"]`, `np.where()` or `pd.cut()`, runs one compiled loop over the whole array. `df.apply(func, axis=1)` does the opposite. It builds a Series for each row and calls your Python function once per row.

The gap is large. On one million rows, a row-wise `apply` that picks a weight when a flag is set took 4.0 seconds. `np.where` did the same job in 7.5

milliseconds, about 500 times faster. (pandas 3.0.6, NumPy 2.4.6, one laptop core.)

Speed is not the only gain. Vectorised code states the rule for the whole column in one line, so a reviewer checks one expression instead of a branching function.

Reach for these first:

Task	Vectorised tool
Arithmetic between columns	<code>+, -, *, /</code> on Series
Two-way choice	<code>np.where(cond, a, b)</code>
Many-way choice	<code>np.select(conditions, choices, default)</code>
Numeric bands	<code>pd.cut(values, bins, labels, right=False)</code>
Lookup from a dict	<code>Series.map(mapping)</code>
Limits	<code>Series.clip(lower, upper)</code>

Watch missing values. `NaN > 60` is `False`, so `np.where(minutes > 60, "heavy", "light")` labels a missing value "light". Handle missing values on purpose: `pd.cut` keeps `NaN` as missing, and `Series.where(values.notna())` puts it back after `np.where`.

Keep `apply` for work with no vectorised form, such as calling an external parser on each string. Even then, try `Series.map` or a list comprehension on one column before `apply(axis=1)`.

Why it matters

A weighting script derives 25 columns with `apply(axis=1)`. It ran in 8 minutes on 2 million rows, so nobody noticed. The programme then merged three trackers into one file of 40 million rows. At the measured rate of about 4 seconds per million rows per column, the derivations alone need roughly 67 minutes. The job hit its 60-minute timeout every night for a week, and weighted results for all three trackers were late.

The same script also labelled respondents with a missing `minutes` value as "light" viewers, because the row function used `if row["minutes"] > 60`. The vectorised rewrite fixed the speed, and review of the one-line rule exposed the missing-value bug.

Worked example: weak code

```
1 import pandas as pd
2
```

```

3
1 def derive(df):
5     def per_row(row):
6         weighted_reach = row["final_weight"] if row["reach_flag"] == 1
           else 0.0
2         if row["age"] < 35:
8             band = "16-34"
9             elif row["age"] < 55:
10                band = "35-54"
11            else:
12                band = "55+"
13            return pd.Series({"weighted_reach": weighted_reach, "age_group":
                band})
14
4     df[["weighted_reach", "age_group"]] = df.apply(per_row, axis=1)
16     return df

```

- Line 4.** No type hints and no docstring. The reader cannot tell what the frame must contain.
- Line 7.** `NaN < 35` is False, so a missing age falls through to `55+` without a warning.
- Line 15.** Builds a Series and calls Python for every row. About 500 times slower than a vectorised rule.
- Line 15.** Mutates the caller's frame. A pure function returns a new frame.

Worked example: strong code

```

1 import numpy as np
2 import pandas as pd
3
1 AGE_BINS = [16, 35, 55, np.inf]
5 AGE_LABELS = ["16-34", "35-54", "55+"]
6
7
8 def derive(df: pd.DataFrame) -> pd.DataFrame:
9     """Add weighted reach and age group.
10
11     Args:
12         df: Respondents with reach_flag, final_weight and age.
13
14     Returns:
15         A new frame. Missing ages give a missing age_group.
16     """
2     return df.assign(
3         weighted_reach=np.where(df["reach_flag"] == 1,
17         df["final_weight"], 0.0),
4         age_group=pd.cut(df["age"], bins=AGE_BINS, labels=AGE_LABELS,
18         right=False),

```

1. **Line 4.** Band edges live in one constant, so a change to the bands is a one-line change.
2. **Line 17.** `assign` returns a new frame, so the caller's data is untouched.
3. **Line 18.** One compiled pass over the column.
4. **Line 19.** `right=False` makes each band include its lower edge: 35 goes to 35-54. Missing ages stay missing.

The strong version is shorter, about 500 times faster on large frames, and handles missing ages on purpose.

Recap card

- **Why is `df.apply(func, axis=1)` slow?** It calls a Python function once per row. `np.where` measured about 500 times faster on one million rows.
- **Vectorised if-else in NumPy** `np.where(cond, value_if_true, value_if_false)`. For several conditions, `np.select`.
- **What does `NaN > 60` return?** False. `np.where` and `np.select` send missing values to a real category unless you handle them.
- **Numeric bands that keep missing values missing** `pd.cut(values, bins, labels, right=False)`.

Practice

Exercise 1. Multiple choice (level 1)

Which line computes `weighted_reach` for a frame with one million rows in a single compiled pass?

- a) `df.apply(lambda r: r["final_weight"] if r["reach_flag"] == 1 else 0.0, axis=1)`
- b) `[w if f == 1 else 0.0 for w, f in zip(df["final_weight"], df["reach_flag"])]`
- c) `np.where(df["reach_flag"] == 1, df["final_weight"], 0.0)`
- d) `for i, row in df.iterrows(): ...`

Why?

- r1) It evaluates the condition and the choice on whole arrays in compiled code.
- r2) It uses fewer characters than the other options.
- r3) NumPy runs the lambda on several cores.

Exercise 2. Predict the output (level 2)

What does this print?

```
1 import pandas as pd
2
3 s = pd.Series([0.0, 1.5, None])
4 print((s > 1).tolist())
```

Exercise 3. Fill the gap (level 2)

Complete the line so rows with `reach_flag == 1` keep their weight and every other row gets zero.

```
1 df["weighted_reach"] = np.__(1)(df["reach_flag"] == 1,
    df["final_weight"], __(2))
```

Exercise 4. Write the function (level 3)

Write `weighted_reach_pct(df)`. It returns the weighted share of rows with `reach_flag == 1`: the sum of `final_weight` where the flag is 1, divided by the sum of all `final_weight`. Return `None` when the total weight is zero or the frame is empty. Do not use `apply` or a loop over rows.

Starter code:

```
1 import numpy as np
2 import pandas as pd
3
4
5 def weighted_reach_pct(df: pd.DataFrame) -> float | None:
6     ...
```

Exercise 5. Refactor (level 4)

Refactor `add_age_group` to vectorised code. Keep the bands: under 35 is 16-34, under 55 is 35-54, anything else is 55+. A missing age must give a missing age group, not 55+. Do not change the input frame.

Starter code:

```
1 import pandas as pd
2
3
4 def add_age_group(df):
5     def band(row):
6         if row["age"] < 35:
7             return "16-34"
8         if row["age"] < 55:
9             return "35-54"
10        return "55+"
11
12    df["age_group"] = df.apply(band, axis=1)
```

```
13     return df
```

Exercise 6. Spot the bug (level 5)

The docstring promises that missing minutes stay missing. A respondent with no minutes recorded shows as "light". Tap the line with the bug, then fix the function.

```
1 def add_viewing_band(df: pd.DataFrame) -> pd.DataFrame:
2     """Band weekly minutes into heavy (60 or more) and light. Missing
   minutes stay missing."""
3     minutes = df["minutes"]
4     band = np.where(minutes >= 60, "heavy", "light")
5     return df.assign(viewing_band=band)
```

Write down the line number of the bug. Then write the fix.

Sources

- Row-wise apply was about 500 times slower than np.where on one million rows. (verified: Measured in this repo's build environment: pandas 3.0.6, NumPy 2.4.6, 4.03 s against 7.5 ms.)
- pd.cut keeps NaN as missing and supports right=False. (verified: Run in pandas 3.0.6.)

B3-L3 Method chaining and Copy-on-Write

Concept

Copy-on-Write (CoW) is always on in pandas 3.0. Every indexing result, such as `df[cols]`, `df[mask]` or `df["a"]`, behaves as a separate object. Changing it never changes `df`. pandas copies data only when you write to a shared object, so reads stay cheap.

One consequence: **chained assignment never works**.

```
1 df[df["age"] < 18]["weight"] = 0      # df unchanged,
   ChainedAssignmentError warning
2 df.loc[df["age"] < 18, "weight"] = 0  # one step: df changed
```

The first line writes to a temporary copy, which Python then discards. pandas 3 emits a `ChainedAssignmentError` warning, but the code keeps running. `SettingWithCopyWarning` no longer exists.

CoW rewards a second habit: **method chaining**. Build the result as one expression of steps that each return a new frame.

```
1 result = (
2     raw
```

```

3     .rename(columns=COLUMN_MAP)
4     .loc[lambda d: d["age"] >= 18]
5     .assign(weight=lambda d: d["weight"].clip(upper=5.0))
6     .pipe(add_age_band, bins=BINS, labels=LABELS)
7 )

```

- `assign(col=lambda d: ...)` refers to the frame at that step, not the original `raw`.
- `.loc[lambda d: ...]` filters on the current step.
- `.pipe(func, **kwargs)` calls your own pure function (lesson B2-L1) inside the chain.

Chains read top to bottom, leave no half-finished variables (`df2`, `df_tmp`, `df_final`) in the namespace, and do not mutate the input. Keep each chain to about ten steps. Split longer ones into named functions and join them with `pipe`.

To debug a chain, comment out steps from the bottom up, or add `.pipe(lambda d: print(d.shape) or d)` between steps.

Why it matters

An analyst caps extreme weights with `df[df["weight"] > 5]["weight"] = 5`. On pandas 2 it printed a `SettingWithCopyWarning` that everyone had learned to ignore. The weights were never capped. Three respondents with weights above 12 moved the reported brand share by 4 points. After the upgrade to pandas 3 the assignment still does nothing, with a different warning.

`df.assign(weight=df["weight"].clip(upper=5.0))` or a single `df.loc[mask, "weight"] = 5` does what the analyst meant.

Worked example: weak code

```

1 df = raw.rename(columns=COLUMN_MAP)
2 df2 = df[df["age"] >= 18]
3 df2[df2["weight"] > 5]["weight"] = 5.0
4 df3 = add_age_band(df2, BINS, LABELS)
5 result = df3

```

1. **Line 2.** Intermediate names that outlive their use and invite edits to the wrong one.
2. **Line 3.** Chained assignment: writes to a temporary copy. The weights are not capped.
3. **Line 5.** Which of `df`, `df2`, `df3` is the final state? The reader has to trace every line.

Worked example: strong code

```
1 import pandas as pd
2
3
4 def prepare(raw: pd.DataFrame, column_map: dict[str, str], bins:
    list[int], labels: list[str]) -> pd.DataFrame:
5     """Rename, keep adults, cap weights at 5 and add age bands."""
6     return (
7         raw
8         .rename(columns=column_map)
9         .loc[lambda d: d["age"] >= 18]
10        .assign(weight=lambda d: d["weight"].clip(upper=5.0))
11        .pipe(add_age_band, bins=bins, labels=labels)
12    )
```

1. **Line 9.** Filters the renamed frame at this step.
2. **Line 10.** Returns a new frame with capped weights. No assignment to a copy.
3. **Line 11.** Your own pure function, called as one step of the chain.

Under Copy-on-Write, write with `.loc` in one step, or better, build a new frame with a chain of `assign`, `loc` and `pipe`.

Recap card

- **Copy-on-Write in pandas 3.0** Always on. Indexing results behave as separate objects, copied only on write.
- **`df[mask]["col"] = value` on pandas 3** Changes nothing and emits `ChainedAssignmentError`. Use `df.loc[mask, "col"] = value`.
- **Refer to the current step inside a chain** A `lambda`: `assign(x=lambda d: ...)` or `loc[lambda d: ...]`.
- **Call your own function inside a chain** `.pipe(func, **kwargs)`.

Practice

Exercise 1. Multiple choice (level 1)

On pandas 3.0, `df[df["age"] < 18]["weight"] = 0` runs. What happens to `df`?

- a) Weights of under-18s become 0.
- b) Nothing. pandas emits a `ChainedAssignmentError` warning.
- c) It raises an exception and stops.
- d) Every weight becomes 0.

Why?

- r1) The first indexing step returns a new object, so the write lands on a temporary copy.

- r2) pandas 3 makes every DataFrame read-only.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3.0?

```
1 import pandas as pd
2
3 df = pd.DataFrame({"a": [1, 2, 3]})
4 s = df["a"]
5 s.iloc[0] = 100
6 print(df["a"].tolist())
```

Exercise 3. Put in order (level 2)

Put the steps of the chain in order. The raw columns are `wt` and `age`. Weights must average 1.0 across the adults in the output.

- `.pipe(add_age_band, bins=BINS, labels=LABELS)`
- `.assign(weight=lambda d: d["weight"] / d["weight"].mean())`
- `.rename(columns={"wt": "weight"})`
- `.loc[lambda d: d["age"] >= 18]`

Exercise 4. Refactor (level 3)

Rewrite `cap_weights` as one method chain that returns a new frame. Keep only rows where `age >= 18`, then cap `weight` at `cap`. Leave the input unchanged. No chained assignment.

Starter code:

```
1 def cap_weights(df, cap):
2     adults = df[df["age"] >= 18]
3     adults[adults["weight"] > cap]["weight"] = cap
4     return adults
```

Exercise 5. Spot the bug (level 4)

Weights for invalid interviews should be zero. The check after the function shows they are unchanged. Tap the line to fix, and rewrite the function.

```
1 import pandas as pd
2
3
4 def zero_invalid(df: pd.DataFrame) -> pd.DataFrame:
5     """Set weight to 0 where valid is False."""
6     out = df.copy()
7     out[~out["valid"]]["weight"] = 0.0
8     return out
```

Write down the line number of the bug. Then write the fix.

Sources

- Copy-on-Write is always enabled in pandas 3.0 and the `mode.copy_on_write` option has no effect. (verified: pandas 3.0 release notes; Pandas4Warning text observed on pandas 3.0.6 and 3.0.2 in this repository.)
- Chained assignment never updates the original in pandas 3.0 and emits `ChainedAssignmentError` as a warning. (verified: pandas Copy-on-Write user guide; run on pandas 3.0.6 and 3.0.2 in this repository.)
- `SettingWithCopyWarning` no longer exists in pandas 3.0. (verified: pandas 3.0 release notes; `pandas.errors` has no `SettingWithCopyWarning` on 3.0.6.)

B4 Polars and DuckDB

Roadmap: Prerequisite for all months. Tools for larger local datasets.

B4-L1 Polars expressions and lazy frames

Concept

Polars is a DataFrame library written in Rust. It runs on all your cores and has no row index. You describe work with **expressions**:

`pl.col("weight").sum()` is a recipe, not a result. Polars runs it inside a context.

```
1 import polars as pl
2
3 shares = (
4     df.group_by("wave", "brand")
5     .agg(pl.col("weight").sum().alias("brand_weight"))
6     .with_columns(total=pl.col("brand_weight").sum().over("wave"))
7 )
```

- `select` returns only the expressions you list. `with_columns` adds or replaces columns. `filter` keeps rows. `group_by(...).agg(...)` aggregates.
- `.over("wave")` computes a group total and broadcasts it back to each row, like a window function in SQL.
- `pl.when(...).then(...).otherwise(...)` replaces `np.where`.

Polars has two modes. A `DataFrame` runs each step at once. A `LazyFrame`, from `df.lazy()` or `pl.scan_parquet(path)`, records the steps and runs nothing until `.collect()`. Before running, the optimiser rewrites the plan. It pushes filters down to the scan, so rows you drop are never read, and reads only the columns you use. `lf.explain()` prints the plan.

Two behaviours to know (Polars 1.33):

- `sum()` **of a group with only nulls returns 0**, not null. Check `null_count()` before you trust a total.
- `mean()` **skips nulls**. `sum() / pl.len()` counts null rows in the denominator and gives a different answer.

Use lazy mode for files. Scan, filter, select, aggregate, then collect once at the end.

Why it matters

A weekly job reads a 9 GB Parquet extract with `pl.read_parquet`, then filters to the current wave. Memory peaks at 14 GB and the job fails on a 16 GB runner on the week the extract grows.

```
pl.scan_parquet(path).filter(pl.col("wave") ==  
wave).select(cols).collect()
```

 lets Polars skip other waves' row groups and unused columns at read time, so memory scales with one wave, not the whole file.

Worked example: weak code

```
❶ df = pl.read_parquet("responses.parquet")  
2 df = df.filter(pl.col("wave") == "2026-09")  
3 totals = df.group_by("brand").agg(pl.col("weight").sum())  
❷ totals = totals.with_columns(mean_w=pl.col("weight") / df.height)
```

1. **Line 1.** Eager read: every wave and every column loads before the filter.
2. **Line 4.** Divides a brand total by the whole wave's row count: not the brand mean, and null rows count.

Worked example: strong code

```
1 import polars as pl  
2  
3  
4 def brand_weights(path: str, wave: str) -> pl.DataFrame:  
5     """Weighted totals and means per brand for one wave. Grain: one row  
   per brand."""  
6     return (  
❶         pl.scan_parquet(path)  
❷         .filter(pl.col("wave") == wave)  
9         .group_by("brand")  
10        .agg(  
11            total=pl.col("weight").sum(),  
12            mean_w=pl.col("weight").mean(),  
❸            missing=pl.col("weight").null_count(),  
14        )  
15        .sort("brand")  
❹        .collect()  
17    )
```

1. **Line 7.** Lazy scan: nothing is read yet.
2. **Line 8.** Pushed down to the scan: other waves are skipped at read time.
3. **Line 13.** Reports nulls, because an all-null group sums to 0.
4. **Line 16.** One collect at the end runs the optimised plan.

Scan lazily, filter and select early, aggregate with expressions, and collect once.

Recap card

- **Polars eager vs lazy** DataFrame runs each step at once. LazyFrame (scan_*, .lazy()) builds a plan and runs on collect().
- **Group total on every row in Polars** pl.col("x").sum().over("group").
- **Polars sum() of an all-null group** 0, not null. Report null_count() with totals.
- **Mean that skips nulls in Polars** pl.col("x").mean(). sum() / pl.len() counts null rows.

Practice

Exercise 1. Multiple choice (level 1)

`lf = pl.scan_parquet("big.parquet").filter(pl.col("wave") == "w9")`. What has happened after this line runs?

- a) The whole file is in memory, filtered to wave w9.
- b) Nothing is read yet. lf holds a query plan.
- c) Only the w9 rows are in memory.
- d) Polars wrote a filtered copy of the file.

Why?

- r1) A LazyFrame records steps and runs them only on collect().
- r2) Parquet files cannot be read without DuckDB.

Exercise 2. Predict the output (level 2)

What does this print on Polars 1.33?

```
1 import polars as pl
2
3 df = pl.DataFrame({"wave": ["w1", "w1", "w2"], "w": [1.0, 2.0, None]})
4 out = df.group_by("wave").agg(pl.col("w").sum()).sort("wave")
5 print(out["w"].to_list())
```

Exercise 3. Fill the gap (level 2)

Add a `share` column with each brand's weight over its wave's total, then run the lazy plan.

```
1 out = (
2     lf.with_columns(share=pl.col("weight") /
3                     pl.col("weight").sum().____(1)("wave"))
4     .____(2)()
5 )
```

Exercise 4. Write the function (level 3)

Write `brand_share(df: pl.DataFrame) -> pl.DataFrame`. Input columns: `wave`, `brand`, `weight`. Return one row per wave and brand with columns `wave`, `brand`, `share`, where `share` is the brand's weight total over the wave's weight total. When a wave's total is 0, `share` is null. Sort by `wave`, then `brand`.

Starter code:

```
1 import polars as pl
2
3
4 def brand_share(df):
5     ...
```

Exercise 5. Spot the bug (level 4)

The mean satisfaction score per region looks too low in regions with many skipped answers. Tap the line with the bug and fix it.

```
1 import polars as pl
2
3
4 def mean_score(df: pl.DataFrame) -> pl.DataFrame:
5     """Mean score per region among respondents who answered."""
6     return (
7         df.group_by("region")
8         .agg(mean_score=pl.col("score").sum() / pl.len())
9         .sort("region")
10    )
```

Write down the line number of the bug. Then write the fix.

Sources

- Polars `sum()` of a group with only nulls returns 0. (verified: Run on Polars 1.33.1 and 1.44.2 in this repository.)
- LazyFrame optimisation pushes filters and column selection down to the scan. (verified: Polars user guide, lazy API and optimisations; plan checked with `explain()` on Polars 1.33.1.)

B4-L2 DuckDB, SQL over files and DataFrames

Concept

DuckDB is an analytical SQL database that runs inside your Python process. There is no server to install. It reads Parquet, CSV and JSON files directly, and it queries pandas and Polars DataFrames that are in scope.

```
1 import duckdb
```

```

2
3 con = duckdb.connect() # in-memory database
4 rel = con.sql("""
5     SELECT wave, brand, sum(weight) AS total
6     FROM read_parquet('extracts/*.parquet')
7     GROUP BY wave, brand
8 """)
9 df = rel.df() # pandas; .pl() for Polars, .fetchall() for tuples

```

- A glob such as `'extracts/*.parquet'` reads many files as one table. Filters and column lists are pushed down to the reader, as in Polars lazy mode.
- `FROM responses` finds a DataFrame variable named `responses` in the calling scope. `con.register("responses", df)` names it explicitly, which is clearer inside functions.
- Results come back as a relation. Nothing is materialised until you call `.df()`, `.pl()`, `.fetchall()` or `.fetchone()`.

Pass values as parameters, never with f-strings.

```

1 con.execute("SELECT count(*) FROM responses WHERE wave = ?", [wave])

```

With `f"... WHERE wave = '{wave}'"`, a value that holds a quote changes the query. In this course's own test, the input `w1` or `'1'='1` turned a filter for one wave into a count of every row. A `?` placeholder (or `$name` with a dict) sends the value as data, so quotes cannot change the SQL. File paths and table names cannot be parameters. Check those against a list of allowed values.

Create the connection at the edge of your program and pass `con` to functions. The module-level `duckdb.sql()` uses one shared default connection, a global that tests cannot replace.

Why it matters

A dashboard helper built its filter with `f"WHERE region = '{region}'"`. The region list came from a client file. One entry was `Hawke's Bay`. The apostrophe broke the SQL, and the job failed at 06:00 before the client's morning meeting. A second entry with a crafted value could have read other clients' rows.

`con.execute("... WHERE region = ?", [region])` handles `Hawke's Bay` as a plain string.

Worked example: weak code

```

1 import duckdb
2
3

```

```

4 def wave_total(wave):
1   q = f"SELECT sum(weight) FROM read_parquet('data/*.parquet') WHERE
    wave = '{wave}'"
2   return duckdb.sql(q).fetchone()[0]

```

1. **Line 5.** Value pasted into SQL: a quote in wave breaks or rewrites the query.
2. **Line 6.** Shared default connection, and None when no rows match.

Worked example: strong code

```

1 import duckdb
2
3 TOTAL_SQL = "SELECT sum(weight) FROM responses WHERE wave = ?"
4
5
6 def wave_total(con: duckdb.DuckDBPyConnection, wave: str) -> float:
7     """Total weight for one wave, 0.0 when the wave has no rows."""
8     row = con.execute(TOTAL_SQL, [wave]).fetchone()
9     if row is None or row[0] is None:
10         return 0.0
11     return float(row[0])

```

1. **Line 3.** The value goes in as a parameter. The SQL text never changes.
2. **Line 6.** The connection is passed in, so a test gives it a small in-memory table.
3. **Line 9.** SQL sum over no rows is NULL. Test with `is None`.

One connection passed in, parameters for values, and a None check on aggregates.

Recap card

- **Query many Parquet files in DuckDB** `FROM read_parquet('dir/*.parquet')`.
- **Pass a value into DuckDB SQL** `con.execute("... WHERE x = ?", [value])`. Never an f-string.
- **Table names from user input** Placeholders cannot bind identifiers. Check the name against an allowlist.
- **SQL sum over no rows** NULL, returned as None. Check with `is None`.

Practice

Exercise 1. Multiple choice (level 1)

Which call safely filters by a `region` value read from a client file?

- a) `con.execute(f"... WHERE region = '{region}')`
- b) `con.execute("... WHERE region = ?", [region])`
- c) `con.execute("... WHERE region = ' + region + '")`

- d) `con.execute("... WHERE region = '%s' " % region)`

Why?

- r1) The value is sent as data, separate from the SQL text.
- r2) It is shorter.

Exercise 2. Predict the output (level 2)

What does this print? (DuckDB 1.5)

```
1 import duckdb
2
3 con = duckdb.connect()
4 con.execute("CREATE TABLE t AS SELECT * FROM (VALUES ('w1', 1.0), ('w2',
    2.0)) v(wave, weight)")
5 print(con.execute("SELECT sum(weight) FROM t WHERE wave =
    'w9'").fetchone())
```

Exercise 3. Fill the gap (level 2)

Count rows for one wave from all Parquet files in `extracts/`, with the wave as a parameter.

```
1 con.execute(
2     "SELECT count(*) FROM ____ (1)('extracts/*.parquet') WHERE wave =
    ____ (2)",
3     [wave],
4 ).fetchone()
```

Exercise 4. Write the function (level 3)

Write `region_total(con, region)`. It returns the sum of `weight` in table `responses` for the given `region`, as a `float`. Return `0.0` when no rows match. Pass `region` as a parameter. The checker runs it with regions such as `Hawke's Bay`.

Starter code:

```
1 import duckdb
2
3
4 def region_total(con, region):
5     return con.execute(f"SELECT sum(weight) FROM responses WHERE region =
    '{region}'").fetchone()[0]
```

Exercise 5. Architecture design (level 4)

A reporting script lets users pick a table to summarise and a wave to filter on. Choose how each input reaches DuckDB.

Options: A ? parameter, An f-string inside the SQL.

1. The wave value

2. The table name
3. The connection

Sources

- DuckDB binds ? and \$name parameters with con.execute(sql, params). (verified: DuckDB Python API documentation; run on DuckDB 1.5.1 and 1.5.5 in this repository.)
- DuckDB queries pandas DataFrames in the calling scope by variable name and reads Parquet globs with read_parquet. (verified: DuckDB Python API documentation; run on DuckDB 1.5.1 and 1.5.5 in this repository.)
- Pasting w1' or '1'='1 into an f-string filter returned every row; the same value as a parameter matched none. (verified: Run on DuckDB 1.5.1 in this repository.)

B4-L3 Choosing pandas, Polars, DuckDB or Spark

Concept

All four tools answer the same questions. Pick by where the data lives and how big it is next to your memory, not by fashion.

A measurement from this course's build: a group-by with sum and mean over 10 million rows and 1,000 groups, best of three runs on a 4-core machine. pandas 3.0.6 took 0.22 s, Polars 1.44 took 0.11 s, DuckDB 1.5 took 0.06 s. At this size every tool is fast enough. The choice matters when data outgrows memory.

Situation	Good default
Data fits comfortably in memory, you need the scikit-learn, statsmodels or plotting ecosystem	pandas
Files larger than you want in memory, heavy transforms, one machine	Polars lazy (scan_parquet)
SQL is the clearest way to express it, or you join many files ad hoc	DuckDB
Data already in a lakehouse, or larger than one machine, or a scheduled production job	Spark (Track A)

The tools share the **Apache Arrow** memory format, so you mix them at the boundaries:

```
1 lazy_result = pl.scan_parquet("extracts/  
    *.parquet").filter(...).group_by(...).agg(...)
```

```

2 summary = lazy_result.collect().to_pandas()      # Polars to pandas for
statsmodels
3 rel = con.sql("SELECT ... FROM read_parquet('extracts/*.parquet')")
4 top = rel.pl()                                  # DuckDB to Polars

```

Rules for mixing:

- **Reduce first, convert last.** Filter and aggregate in the engine that reads the files. Convert the small result.
- **Watch the index.** Polars has no index. `pl.from_pandas(df)` drops a pandas index unless you pass `include_index=True`, or call `reset_index()` first.
- **Check dtypes after each hop.** Text becomes `str` in pandas 3 and `String` in Polars. Nullable integers need care in both directions.

One project, one primary engine. A second tool at a clear boundary is fine. Four tools in one function is not.

Why it matters

An analyst sets `respondent_id` as the pandas index to deduplicate, then hands the frame to a Polars function for a fast join. `pl.from_pandas` drops the index. The join key is gone, so the analyst joins on row position with `hstack`. Two waves sorted differently, and 11% of respondents got another person's scores.

`pl.from_pandas(df.reset_index())` keeps the ID as a column, and the join uses it.

Worked example: weak code

```

❶ big = pl.read_parquet("extracts/*.parquet").to_pandas()
❷ big = big[big["wave"] == WAVE].set_index("respondent_id")
❸ stats = pl.from_pandas(big).group_by("brand").agg(pl.col("weight").sum())

```

1. **Line 1.** Reads every wave eagerly, then copies all of it into pandas before filtering.
2. **Line 2.** The ID moves into the index.
3. **Line 3.** Back to Polars: the index, and so the ID, is dropped. Two conversions of the full data.

Worked example: strong code

```

1 import pandas as pd
2 import polars as pl
3
4
5 def brand_totals(path_glob: str, wave: str) -> pd.DataFrame:

```

```

6      """Weight per brand for one wave, as pandas for modelling. Grain: one
      row per brand."""
7      return (
8          pl.scan_parquet(path_glob)
9          .filter(pl.col("wave") == wave)
10         .group_by("brand")
11         .agg(total=pl.col("weight").sum())
12         .sort("brand")
13         .collect()
14         .to_pandas()
15     )

```

1. **Line 9.** Reduce in the engine that reads the files.
2. **Line 14.** Convert once, at the end, when the result has one row per brand.

One primary engine. Reduce where the data is read. Convert small results at a clear boundary.

Recap card

- **Group-by over 10M rows: pandas, Polars, DuckDB** 0.22 s, 0.11 s, 0.06 s on 4 cores. All fast enough. Choose by size against memory and where data lives.
- **Mixing engines** Reduce first in the engine that reads the files, convert the small result last.
- **pl.from_pandas and the index** Dropped by default. `reset_index()` first, or `include_index=True`.
- **When Spark** Data already in the lakehouse, larger than one machine, or a scheduled production job.

Practice

Exercise 1. Multiple choice (level 1)

You have 40 GB of Parquet on a laptop with 16 GB of RAM. You need one table of weighted totals by wave and brand, once. Which is the best first choice?

- a) pandas `read_parquet`, then `groupby`
- b) DuckDB or Polars lazy, aggregating while reading
- c) Set up a Spark cluster
- d) Split the files by hand and run pandas on each

Why?

- r1) Both read Parquet in chunks and push filters and aggregation into the scan, so the full data never sits in memory.

- r2) They are newer than pandas.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0, Polars 1.33)

```
1 import pandas as pd
2 import polars as pl
3
4 pdf = pd.DataFrame({"respondent_id": ["r1", "r2"], "score": [3,
    4]}).set_index("respondent_id")
5 print(pl.from_pandas(pdf).columns)
```

Exercise 3. Fill the gap (level 2)

Hand a DuckDB result to Polars, then a Polars result to pandas.

```
1 top_brands = con.sql("SELECT brand, sum(weight) AS total FROM responses
    GROUP BY brand").____(1)()
2 for_model = top_brands.____(2)()
```

Exercise 4. Spot the bug (level 3)

This function runs out of memory on the full extract. Tap the line that loads far more than the result needs, and fix the function.

```
1 import pandas as pd
2 import polars as pl
3
4
5 def wave_mean(path: str, wave: str) -> float:
6     """Mean weight for one wave."""
7     df = pl.read_parquet(path).to_pandas()
8     return float(df.loc[df["wave"] == wave, "weight"].mean())
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 4)

Choose the primary engine for each job.

Options: pandas, Polars lazy, DuckDB, Spark.

1. Fit a logistic regression with statsmodels on a 50,000-row survey file
2. Ad hoc SQL joins across 30 CSV and Parquet exports on your laptop
3. Nightly job over tables that already live in the Databricks lakehouse
4. Heavy column transforms over 25 GB of Parquet on one 32 GB machine

Sources

- Group-by sum and mean over 10M rows: pandas 3.0.6 0.22 s, Polars 1.44.2 0.11 s, DuckDB 1.5.5 0.06 s (best of 3, 4 cores). (verified: Measured in this repository's dev environment.)
- `pl.from_pandas` drops the pandas index unless `include_index=True`. (verified: Polars API reference; run on Polars 1.33.1 and 1.44.2 in this repository.)

B5 Data validation

Roadmap: Prerequisite for all months. Schema checks with pandera and input models with pydantic.

B5-L1 DataFrame schemas with pandera

Concept

A **schema** states what a DataFrame must look like: which columns exist, their dtypes, and rules on their values. **pandera** checks a pandas DataFrame against a schema and raises when it does not match. Examples here use pandera 0.33 with pandas 3.

```
1 import pandera.pandas as pa
2
3 RESPONSES = pa.DataFrameSchema(
4     {
5         "respondent_id": pa.Column(str, unique=True),
6         "age": pa.Column(int, pa.Check.in_range(16, 99)),
7         "weight": pa.Column(float, pa.Check.gt(0)),
8     },
9     strict=True,
10    coerce=True,
11 )
12 clean = RESPONSES.validate(raw, lazy=True)
```

- `pa.Column(dtype, checks, nullable=False)` is the default: nulls fail unless you set `nullable=True`.
- `unique=True` rejects duplicate keys.
- `strict=True` rejects columns the schema does not name. `coerce=True` converts dtypes before checking.
- `lazy=True` collects **every** failure into one `SchemaErrors` exception. Its `failure_cases` attribute is a DataFrame with the column, check and failing value. Without `lazy`, validation stops at the first `SchemaError`.

`SchemaErrors` is not a subclass of `SchemaError`. Catch both when you handle failures. A `strict=True` violation raises `SchemaErrors` even without `lazy=True`.

The class syntax, `pa.DataFrameModel`, does the same with type hints. Add `@pa.check_types` to a function whose parameters are annotated `DataFrame[Model]`, and pandera validates inputs and outputs on every call.

Import from `pandera.pandas`. The plain `import pandera` still works in 0.33, but the pandas-specific module is the documented path.

pandera does not run in this app's browser runtime. The exercises check your code's structure. Run `pytest` locally with the `dev` group to execute schemas for real.

Why it matters

A supplier changes a weight export so that non-responders get weight `0` instead of being left out. The weighting script divides by the sum of weights per cell. Three cells now contain only zeros. The script writes `NaN` shares to the client dashboard, and nobody sees an error.

`pa.Column(float, pa.Check.gt(0))` on the input fails the run on day one, with the failing rows listed in `failure_cases`.

Worked example: weak code

```
1 def load_weights(path):
2     df = pd.read_csv(path)
3     ❶ assert "weight" in df.columns
4     ❷ df = df.dropna()
5     return df
```

1. **Line 3.** Checks one column exists. Not its type, its values or other columns. And python -O strips asserts.
2. **Line 4.** Silently drops rows with any null, in any column. The count of dropped rows is never reported.

Worked example: strong code

```
1 import pandas as pd
2 import pandera.pandas as pa
3
4 WEIGHTS = pa.DataFrameSchema(
5     {
6     ❶     "respondent_id": pa.Column(str, unique=True),
7         "cell": pa.Column(str),
8     ❷     "weight": pa.Column(float, pa.Check.gt(0)),
9     },
10    ❸    strict=True,
11    coerce=True,
12 )
13
14
15 def load_weights(path: str) -> pd.DataFrame:
16     """Read and validate a weights file. Grain: one row per respondent.
17
18     Raises:
19         pa.errors.SchemaErrors: Listing every failing row and check.
20     """
```

```
4 return WEIGHTS.validate(pd.read_csv(path, dtype={"respondent_id":  
"str"}), lazy=True)
```

1. **Line 6.** The grain, enforced: one row per respondent.
2. **Line 8.** Not null (the default) and positive: zero weights fail loudly.
3. **Line 10.** A new or renamed column in the supplier file fails the run.
4. **Line 21.** `lazy=True` reports every failure at once, not just the first.

A schema at the point of entry turns a silent bad input into one loud, detailed failure.

Recap card

- **pandera Column defaults** `nullable=False`. Set `nullable=True` where nulls are expected.
- **`strict=True` and `coerce=True`** `strict` rejects unknown columns. `coerce` converts dtypes before checks.
- **`validate(df, lazy=True)`** Runs every check and raises `SchemaErrors` with `failure_cases`.
- **`SchemaError` vs `SchemaErrors`** Separate classes. `SchemaErrors` is not a subclass. Catch both, or better, let them raise.

Practice

Exercise 1. Multiple choice (level 1)

In `pandera 0.33`, what does `lazy=True` change in `schema.validate(df, lazy=True)` ?

- a) Validation runs later, when the `DataFrame` is first used.
- b) Every failure is collected and raised together as `SchemaErrors`.
- c) Failing rows are dropped instead of raising.
- d) Only dtypes are checked, not values.

Why?

- r1) You see all problems in one run, with `failure_cases` listing each one.
- r2) It is faster because it skips checks.

Exercise 2. Predict the output (level 2)

What does this print? (`pandera 0.33`)

```
1 import pandas as pd
2 import pandera.pandas as pa
3
4 schema = pa.DataFrameSchema({"w": pa.Column(float, pa.Check.gt(0))})
5 try:
```

```

6     schema.validate(pd.DataFrame({"w": [1.0, -2.0, -3.0]}), lazy=True)
7 except pa.errors.SchemaErrors as err:
8     print(err.failure_cases["failure_case"].tolist())

```

Exercise 3. Fill the gap (level 2)

Reject duplicate IDs, and reject any column the schema does not name.

```

1 schema = pa.DataFrameSchema(
2     {"respondent_id": pa.Column(str, ____ (1)=True), "score":
3     pa.Column(int)},
4     ____ (2)=True,
5 )

```

Exercise 4. Write the function (level 3)

Define a module constant `SCORES` as a pandera `DataFrameSchema` with:

- `respondent_id`: str, unique.
- `nps`: int, between 0 and 10 inclusive.
- `comment`: str, nulls allowed.
- Unknown columns rejected, dtypes coerced.

Then write `validate_scores(df: pd.DataFrame) -> pd.DataFrame` that returns `SCORES.validate(df, lazy=True)`.

pandera does not run in the browser, so the checker reads your code. Run it locally with `pytest` to see it validate.

Starter code:

```

1 import pandas as pd
2 import pandera.pandas as pa
3
4
5 def validate_scores(df):
6     return df

```

Exercise 5. Spot the bug (level 4)

A bad weights file passed through this loader and broke the dashboard. Tap the line that let it through.

```

1 import pandas as pd
2 import pandera.pandas as pa
3
4 WEIGHTS = pa.DataFrameSchema({"weight": pa.Column(float,
5     pa.Check.gt(0))}, strict=True)
6
7 def load_weights(path: str) -> pd.DataFrame:
8     """Read and validate weights."""
9     df = pd.read_csv(path)

```

```

10     try:
11         return WEIGHTS.validate(df, lazy=True)
12     except pa.errors.SchemaError:
13         return df

```

Write down the line number of the bug. Then write the fix.

Sources

- pandera 0.33: lazy=True raises SchemaErrors with a failure_cases DataFrame. (verified: pandera documentation; run on pandera 0.33.1 with pandas 3.0.6 in this repository.)
- SchemaErrors is not a subclass of SchemaError in pandera 0.33. (verified: issubclass check on pandera 0.33.1 in this repository.)
- A strict=True violation raises SchemaErrors even without lazy=True. (verified: Run on pandera 0.33.1 in this repository.)
- import pandera.pandas as pa is the documented import for pandas schemas. (search-only: pandera documentation (docs site not reachable from this environment).)

B5-L2 pydantic models for config and input records

Concept

A dataclass (lesson B2-L2) does not check types. **pydantic** does. A `BaseModel` validates and converts data when you build it, and raises one `ValidationError` that lists every problem. Examples use pydantic 2.12.

```

1  from pydantic import BaseModel, ConfigDict, Field
2
3
4  class WeightingConfig(BaseModel):
5      model_config = ConfigDict(extra="forbid", frozen=True)
6
7      weight_col: str
8      min_base: int = Field(default=50, gt=0)
9      cap: float = Field(default=5.0, gt=1.0)
10
11
12  cfg = WeightingConfig.model_validate(yaml.safe_load(text))

```

- `Field(gt=0, le=10)` adds bounds. `@field_validator("name")` adds your own rule.
- `extra="forbid"` rejects unknown keys, so `minbase: 30` in a YAML file fails instead of being ignored. The default, `"ignore"`, drops them silently.
- `frozen=True` rejects assignment after construction.

- `e.errors()` returns a list of dicts with `loc`, `type` and `msg`. Log it as structured data.

pydantic runs in **lax mode** by default. It converts `"30"` to `30` and `30.0` to `30`, but rejects `"thirty"` and `30.5`. It also converts `True` to `1` for an `int` field. Set `ConfigDict(strict=True)` when inputs must already have the right type.

One gap: `model_copy(update={...})` does **not** validate.

`cfg.model_copy(update={"min_base": -1})` returns a config with `min_base = -1`. To change a validated model, build a new one:

`WeightingConfig.model_validate(**cfg.model_dump(), "min_base": 30)`.

Use pydantic for records and settings that arrive from outside: YAML, JSON, API payloads. Use pandera (lesson B5-L1) for whole DataFrames. For a list of records, `TypeAdapter(list[Model]).validate_python(rows)` validates them in one call.

Why it matters

A weighting config file had `cap: 0.5` typed where `5` was meant, and `minbase` spelt without the underscore. The loader used a plain dict. The cap shrank every weight above `0.5` down to `0.5`, which flattened the weighting, and `min_base` stayed at its default. The report went out with unweighted-looking results.

`WeightingConfig.model_validate` with `gt=1.0` and `extra="forbid"` rejects both mistakes in one error message, before any data is read.

Worked example: weak code

```
1 def load_config(raw):
2     return {
3         "weight_col": raw.get("weight_col", "weight"),
4         "min_base": int(raw.get("min_base", 50)),
5         "cap": float(raw.get("cap", 5.0)),
6     }
```

1. **Line 3.** A missing key falls back to a guess.
2. **Line 4.** A typo such as `minbase` is ignored. `int()` accepts negatives.
3. **Line 5.** No bounds: `0.5` passes.

Worked example: strong code

```
1 from pydantic import BaseModel, ConfigDict, Field
2
3
4 class WeightingConfig(BaseModel):
5     """Settings for one weighting run, loaded from YAML."""
```

```

6
1 model_config = ConfigDict(extra="forbid", frozen=True)
8
2 weight_col: str
10 min_base: int = Field(default=50, gt=0)
3 cap: float = Field(default=5.0, gt=1.0)
12
13
14 def load_config(raw: dict[str, object]) -> WeightingConfig:
15     """Validate a parsed YAML mapping.
16
17     Raises:
18         pydantic.ValidationError: Listing every invalid or unknown key.
19     """
20     return WeightingConfig.model_validate(raw)

```

1. **Line 7.** Unknown keys fail. No changes after loading.
2. **Line 9.** Required: no default, so a missing key fails.
3. **Line 11.** A bound that encodes what a cap means.

Validate external settings and records once, at the boundary, with every rule in the model.

Recap card

- **pydantic extra keys, default behaviour** Ignored silently. Use `ConfigDict(extra="forbid")` for config.
- **pydantic lax mode** Converts "30" and 30.0 to 30, True to 1. Use `strict=True` to refuse conversions.
- **model_copy(update=...)** Does not validate. Rebuild with `model_validate(**m.model_dump(), ...)`.
- **pydantic or pandera** pydantic for records and settings from outside. pandera for whole DataFrames.

Practice

Exercise 1. Multiple choice (level 1)

A pydantic 2 model has the default config. A YAML file adds an unknown key `minbase: 30`. What happens on `model_validate`?

- a) `ValidationError: extra_forbidden`
- b) The key is ignored silently.
- c) `min_base` is set to 30.
- d) A warning is printed.

Why?

- r1) The default extra setting is "ignore". Set extra="forbid" to reject unknown keys.
- r2) pydantic never checks keys.

Exercise 2. Predict the output (level 2)

What does this print? (pydantic 2.12)

```
1 from pydantic import BaseModel, Field
2
3
4 class Cfg(BaseModel):
5     min_base: int = Field(default=50, gt=0)
6
7
8 cfg = Cfg(min_base="30")
9 print(cfg.model_copy(update={"min_base": -1}).min_base)
```

Exercise 3. Fill the gap (level 2)

Reject unknown keys, stop changes after loading, and require nps between 0 and 10.

```
1 class Response(BaseModel):
2     model_config = ConfigDict(extra="____(1)", ____ (2)=True)
3
4     respondent_id: str
5     nps: int = Field(ge=0, ____ (3)=10)
```

Exercise 4. Write the function (level 3)

Write a pydantic model `WeightingConfig` and a function `load_config(raw)`.

- Fields: `weight_col: str` (required), `min_base: int` default 50 and greater than 0, `cap: float` default 5.0 and greater than 1.0.
- Reject unknown keys. Reject assignment after construction.
- `load_config(raw: dict[str, object]) -> WeightingConfig` validates the mapping.

Starter code:

```
1 from pydantic import BaseModel, ConfigDict, Field
2
3
4 def load_config(raw):
5     return raw
```

Exercise 5. Spot the bug (level 4)

A job lowers the minimum base for a small market. The run then reports subgroups of size 0 as valid. Tap the line with the bug and fix the function so the new value is validated.

```
1 from pydantic import BaseModel, ConfigDict, Field
2
3
4 class Cfg(BaseModel):
5     """Run settings."""
6
7     model_config = ConfigDict(extra="forbid", frozen=True)
8
9     min_base: int = Field(default=50, gt=0)
10
11
12 def with_min_base(cfg: Cfg, value: int) -> Cfg:
13     """Return a copy of cfg with a new min_base."""
14     return cfg.model_copy(update={"min_base": value})
```

Write down the line number of the bug. Then write the fix.

Sources

- pydantic 2 default extra behaviour is ignore; extra="forbid" raises extra_forbidden. (verified: pydantic documentation; run on pydantic 2.12.5 and 2.13.5 in this repository.)
- Lax mode converts "30" and 30.0 to int 30 and True to 1, and rejects 30.5 and "thirty". (verified: pydantic conversion table; run on pydantic 2.12.5 in this repository.)
- model_copy(update=...) does not validate the update. (verified: pydantic documentation for model_copy; run on pydantic 2.12.5 in this repository.)

B5-L3 Validate at the boundaries, with severity

Concept

Validation belongs at **boundaries**, where data crosses from one owner to another:

- **On the way in.** After reading a file, API response or config. Check schema, keys and value ranges (lessons B5-L1 and B5-L2).
- **On the way out.** Before writing a table or sending a report. Check the promises you make to the next consumer: grain, row counts, totals, shares that sum to 1.

Checks in the middle of a pure transformation add noise. A function that receives validated input and is unit tested does not need to re-check it.

Not every failed check should stop a run. Give each check a **severity**:

- **CRITICAL** stops the run. Wrong grain, missing required column, empty input, weights that do not sum to the sample size.
- **WARN** records the problem and continues. Missing share above its usual level, a subgroup below its reporting base.

Record results as **structured data**, not prose, so a dashboard or alert filters them:

```
1 Severity = Literal["CRITICAL", "WARN"]
2
3
4 @dataclass(frozen=True)
5 class QaResult:
6     check: str
7     severity: Severity
8     passed: bool
9     observed: float | None
```

Run all checks, collect the results, log them, **then** raise if any CRITICAL check failed. The log keeps every result, even on the run that stops.

Two coding rules make checks trustworthy:

- **Guard every division.** An empty input has no missing share. Return `None`, not a `ZeroDivisionError`.
- **Test `None` with `is None`.** `if share:` treats a real share of `0.0` like a missing one, and the check silently never runs.

Why it matters

A tracker QA step printed "WARNING: low base in 3 cells" to the notebook output and carried on. Nobody reads notebook output for scheduled jobs. The same step never checked for an empty input. The week a supplier delivered an empty file, the job computed shares from zero rows, divided by zero, and wrote `NaN` to every cell of the client dashboard.

Structured results in a table get alerts. A CRITICAL `min_rows` check stops the run before the empty file reaches the dashboard.

Worked example: weak code

```
1 def qa(df):
2     share = df["weight"].isna().sum() / len(df)
3     if share:
4         if share > 0.1:
```

```

3         print("WARNING: many missing weights")
6     if len(df) == 0:
4         print("empty file")

```

1. **Line 2.** Divides by zero on an empty file, before the emptiness check runs.
2. **Line 3.** Truthiness: a share of 0.0 skips the check.
3. **Line 5.** Prose on stdout. Nothing stores or alerts on it.
4. **Line 7.** An empty file is critical, but the run carries on.

Worked example: strong code

```

1  from dataclasses import dataclass
2  from typing import Literal
3
4  import pandas as pd
5
6  Severity = Literal["CRITICAL", "WARN"]
7
8
9  @dataclass(frozen=True)
10 class QaResult:
11     """One check outcome, stored in the run log."""
12
13     check: str
14     severity: Severity
15     passed: bool
16     observed: float | None
17
18
19 def run_checks(df: pd.DataFrame, min_rows: int, max_missing: float) ->
    list[QaResult]:
20     """Evaluate every input check. Never raises."""
21     n = len(df)
22     share = None if n == 0 else float(df["weight"].isna().sum()) / n
23     return [
24         QaResult("min_rows", "CRITICAL", n >= min_rows, float(n)),
25         QaResult("missing_weight", "WARN", share is not None and share <=
    max_missing, share),
26     ]
27
28
29 def enforce(results: list[QaResult]) -> None:
30     """Raise when any CRITICAL check failed."""
31     failed = [r.check for r in results if r.severity == "CRITICAL" and
    not r.passed]
32     if failed:
33         raise RuntimeError(f"critical checks failed: {failed}")

```

1. **Line 6.** A closed set of severities, checked by mypy.
2. **Line 22.** Guarded division: no share on an empty frame.
3. **Line 25.** is None, so a real 0.0 passes the check instead of skipping it.

4. **Line 33.** Raised after all checks ran, so the log has the full picture.

Check at boundaries. Collect structured results. Log them all, then stop on CRITICAL.

Recap card

- **Where to validate** At boundaries: after reading input, before writing output.
- **CRITICAL vs WARN** CRITICAL stops the run (raise). WARN records and continues.
- **Order of QA operations** Run every check, log all results, then raise if any CRITICAL failed.
- **Two coding rules for checks** Guard every division. Test for None with is None, never truthiness.

Practice

Exercise 1. Multiple choice (level 1)

A pipeline reads a supplier file, runs five pure transforms, then writes a client table. Where do validation checks belong?

- a) After every transform.
- b) After reading, and before writing.
- c) Only before writing.
- d) Nowhere, if the transforms have unit tests.

Why?

- r1) Those are the boundaries where data changes owner. Tests cover the pure middle.
- r2) Fewer checks run faster.

Exercise 2. Predict the output (level 2)

No weights are missing this week. What does this print?

```
1 def missing_check(share: float | None, limit: float) -> str:
2     if share:
3         return "pass" if share <= limit else "fail"
4     return "no data"
5
6
7 print(missing_check(0.0, 0.1))
```

Exercise 3. Put in order (level 2)

Order the steps of one pipeline run.

- Run input checks and collect results
- Write the client table
- Run output checks on the result
- Run the pure transforms
- Log all results, then raise if any CRITICAL failed
- Read the supplier file

Exercise 4. Write the function (level 3)

Write `run_checks(df, min_rows, max_missing) -> list[QaResult]` and `enforce(results) -> None`. `QaResult` and `Severity` are given.

- `min_rows`, CRITICAL: passes when the row count is at least `min_rows`. `observed` is the row count as a float.
- `missing_weight`, WARN: the share of null `weight` values. Passes when the share is at most `max_missing`. On an empty frame the share is `None` and the check fails. `observed` is the share.
- `run_checks` never raises. `enforce` raises `RuntimeError` naming each failed CRITICAL check, and ignores WARN failures.

Starter code:

```
1 from dataclasses import dataclass
2 from typing import Literal
3
4 import pandas as pd
5
6 Severity = Literal["CRITICAL", "WARN"]
7
8
9 @dataclass(frozen=True)
10 class QaResult:
11     """One check outcome."""
12
13     check: str
14     severity: Severity
15     passed: bool
16     observed: float | None
17
18
19 def run_checks(df, min_rows, max_missing):
20     ...
21
22
23 def enforce(results):
```

Exercise 5. Architecture design (level 4)

Assign a severity to each check on a weekly brand-tracking run.

Options: CRITICAL, WARN.

1. respondent_id is unique in the input
2. A brand's subgroup base is below 50
3. Weights sum to the sample size within 0.1%
4. Share of missing age is above last month's level

B6 Reproducibility

Roadmap: Prerequisite for all months. Seeds, config, data versioning and pinned dependencies.

B6-L1 Seeds and random generators

Concept

A result you cannot rerun is a result you cannot defend. Randomness enters analysis through sampling, train/test splits, bootstrap intervals, model initialisation and synthetic data. Control each source.

Create a generator and pass it in. NumPy 1.17 introduced

`numpy.random.Generator`. Examples use NumPy 2.4.

```
1 rng = np.random.default_rng(20260925)      # at the edge, once
2 sample = draw_sample(ids, k=500, rng=rng)    # the function receives rng
```

- `np.random.default_rng(seed)` gives the same stream of numbers for the same seed, on every run and machine with the same NumPy version.
- Functions take `rng: np.random.Generator` as a parameter. They never call `np.random.seed()` or the legacy `np.random.rand()`, which share one hidden global state across your whole program and every library you import.
- One generator, used twice, gives two **different** draws. That is correct: two samples should differ. Rerunning the whole script gives the same pair.
- Need independent streams for parallel workers? `rng.spawn(n)` returns `n` child generators.

Libraries take the seed or generator as a parameter too:

```
train_test_split(..., random_state=42) in scikit-learn, df.sample(n,
random_state=rng) in pandas.
```

Two hidden sources of non-determinism:

- `hash()` **on strings changes between Python processes** (hash randomisation, `PYTHONHASHSEED`). `hash(respondent_id) % 10` puts the same respondent in a different bucket on each run. Use `hashlib.sha256` for stable buckets.
- **Set iteration order** for strings also changes between runs. Sort before you rely on order.

Record the seed in the run log with the results, so anyone reruns the exact analysis.

Why it matters

A holdout set was defined as `hash(respondent_id) % 10 == 0`. The model scored AUC 0.81 in the notebook. The scheduled retraining job, a new Python process, put a different 10% in the holdout. Some of the old test respondents were now in training, and the reported AUC rose to 0.84 with no change to the model. The team spent a week chasing an improvement that did not exist.

A bucket from `hashlib.sha256` of the ID is the same in every process.

Worked example: weak code

```
1 import numpy as np
2
3 np.random.seed(1)
4
5
6 def draw_sample(ids, k):
7     return list(np.random.choice(ids, size=k, replace=False))
8
9
10 def holdout(ids):
11     return [i for i in ids if hash(i) % 10 == 0]
```

1. **Line 3.** Global state, set at import: any other call to `np.random` changes what `draw_sample` returns.
2. **Line 7.** Reads the hidden global generator.
3. **Line 11.** String `hash()` is randomised per process: a different holdout every run.

Worked example: strong code

```
1 import hashlib
2
3 import numpy as np
4
5
6 def draw_sample(ids: list[str], k: int, rng: np.random.Generator) -> list[str]:
7     """Draw k distinct IDs using the generator passed in."""
8     return [str(x) for x in rng.choice(ids, size=k, replace=False)]
9
10
11 def bucket(respondent_id: str, n_buckets: int) -> int:
12     """Stable bucket for an ID, the same in every process."""
13     digest = hashlib.sha256(respondent_id.encode("utf-8")).hexdigest()
14     return int(digest, 16) % n_buckets
15
16
17 def main(seed: int) -> None:
18     """The edge: create the generator once and log the seed."""
```

```
3 rng = np.random.default_rng(seed)
20 ...
```

1. **Line 6.** The randomness is a parameter: tests pass a seeded generator.
2. **Line 13.** A cryptographic hash is stable across processes and machines.
3. **Line 19.** One generator, created at the edge from a logged seed.

Seed once at the edge, pass the generator in, and never use `hash()` for anything that must be stable.

Recap card

- **Reproducible randomness in NumPy 2** `rng = np.random.default_rng(seed)`, created once at the edge and passed in.
- **Why not `np.random.seed`?** It seeds one global generator that every library shares and advances.
- **Stable bucket for an ID** `int(hashlib.sha256(id.encode()).hexdigest(), 16) % n`. Never `hash()`, which is randomised per process.
- **Independent streams for workers** `rng.spawn(n)`.

Practice

Exercise 1. Multiple choice (level 1)

Which is the recommended way to get reproducible random numbers in NumPy 2?

- a) `np.random.seed(42)` at the top of the script
- b) `rng = np.random.default_rng(42)`, passed to the functions that need it
- c) `random.seed(42)` from the standard library
- d) No seed. Results average out.

Why?

- r1) Each Generator owns its state, so no other code can change the stream it gives your function.
- r2) `np.random.seed` was removed in NumPy 2.

Exercise 2. Predict the output (level 2)

What does this print? (NumPy 2.4)

```
1 import numpy as np
2
3 rng = np.random.default_rng(1)
4 first = rng.choice(10, size=3, replace=False).tolist()
5 second = rng.choice(10, size=3, replace=False).tolist()
6 print(first == second)
```

Exercise 3. Fill the gap (level 2)

Create a seeded generator and give four parallel workers independent streams.

```
1 rng = np.random.____(1)(20260925)
2 worker_rngs = rng.____(2)(4)
```

Exercise 4. Write the function (level 3)

Write `bucket(respondent_id: str, n_buckets: int) -> int`. It returns a number from 0 to `n_buckets - 1` that is the same for the same ID in every Python process. Use `hashlib.sha256` on the UTF-8 bytes of the ID, read the hex digest as an integer, and take it modulo `n_buckets`. Do not use `hash()`.

Starter code:

```
1 def bucket(respondent_id, n_buckets):
2     return hash(respondent_id) % n_buckets
```

Exercise 5. Spot the bug (level 4)

Two analysts run the same bootstrap with seed 7 and get different confidence intervals. One of them imports a plotting helper that draws random jitter with `np.random`. Tap the line that makes the result depend on other code, and fix the function.

```
1 import numpy as np
2
3
4 def bootstrap_means(values: list[float], n_boot: int, seed: int) ->
    list[float]:
5     """Bootstrap means of values."""
6     np.random.seed(seed)
7     out = []
8     for _ in range(n_boot):
9         draw = np.random.choice(values, size=len(values), replace=True)
10        out.append(float(draw.mean()))
11    return out
```

Write down the line number of the bug. Then write the fix.

Sources

- `np.random.default_rng(seed)` returns a Generator; `Generator.spawn(n)` returns `n` independent child generators. (verified: NumPy random documentation; run on NumPy 2.4.6 in this repository.)
- Python's `hash()` of a `str` differs between processes unless `PYTHONHASHSEED` is fixed. (verified: Python data model documentation (object.**hash**); observed across two processes in this repository.)

B6-L2 Record the data, config and code behind every result

Concept

To rerun a result you need three things exactly as they were: the **data**, the **config** and the **code**. Record all three with every run, in a small **run manifest**.

```
1 manifest = {
2     "run_id": run_id,
3     "started_utc": datetime.now(timezone.utc).isoformat(),
4     "input_sha256": file_sha256(input_path),
5     "config_sha256": config_sha256(config),
6     "git_commit": git_commit,
7     "seed": seed,
8     "packages": {"pandas": version("pandas"), "numpy": version("numpy")},
9 }
```

Data. A file name and a date do not identify content. Suppliers resend files under the same name. A **content hash** does: `hashlib.sha256` over the file bytes changes if any byte changes. `hashlib.file_digest(f, "sha256")` (Python 3.11 and later) reads the file in chunks, so large files do not load into memory. Keep raw inputs immutable: write each delivery to a new path and never overwrite it. For lakehouse tables, record the Delta table version instead (lesson A3-L3).

Config. Hash the config you loaded, not the file on disk, so defaults count. Serialise with `json.dumps(config, sort_keys=True)` first. Without `sort_keys`, the same settings in a different key order give a different hash.

Code. Record the Git commit. Refuse to produce client output from a working tree with uncommitted changes, or record that it was dirty.

Environment. `importlib.metadata.version("pandas")` returns the installed version. The lock file (lesson B6-L3) covers the rest.

Store the manifest next to the output, or as a row in a run log table. When someone asks "why did this number change?", compare two manifests. The field that differs is where to look.

Why it matters

A client queried a brand share that moved 3 points between two versions of the same wave's report. The analysis folder held `wave12.csv`, the config and the notebook, all edited since. Nobody knew whether the supplier had resent

the file, a setting had changed, or the code had. Reconstructing the answer took four days.

With manifests, the two runs differed in one field: `input_sha256`. The supplier had resent the file with 212 late completes.

Worked example: weak code

```
❶ df = pd.read_csv("data/wave12.csv")
  results = analyse(df, CONFIG)
❷ results.to_csv(f"out/wave12_{date.today()}.csv")
```

1. **Line 1.** The name says nothing about the content. A resent file has the same name.
2. **Line 3.** A date, but no record of the input, config, code or seed behind it.

Worked example: strong code

```
1 import hashlib
2 import json
3 from pathlib import Path
4
5
6 def file_sha256(path: Path) -> str:
7     """SHA-256 of a file's bytes, read in chunks."""
8     with path.open("rb") as f:
9         return hashlib.file_digest(f, "sha256").hexdigest()
10
11
12 def config_sha256(config: dict[str, object]) -> str:
13     """SHA-256 of a config, independent of key order."""
14     text = json.dumps(config, sort_keys=True)
15     return hashlib.sha256(text.encode("utf-8")).hexdigest()
```

1. **Line 9.** Content identity: any changed byte changes the hash.
2. **Line 14.** Stable serialisation: key order does not change the hash.

Hash the input and the loaded config, record the commit and the seed, and store the manifest with the output.

Recap card

- **Three things to record for a rerun** Data (content hash or table version), config (hash of the loaded config), code (Git commit).
- **Hash a large file** with `path.open("rb") as f: hashlib.file_digest(f, "sha256")` (Python 3.11+).
- **Stable config hash** `sha256 of json.dumps(config, sort_keys=True).encode("utf-8")`.
- **Raw inputs** Immutable: each delivery at a new path, never overwritten.

Practice

Exercise 1. Multiple choice (level 1)

Which value best identifies the exact input file a result came from?

- a) The file name
- b) The file's modified time
- c) The SHA-256 of the file's bytes
- d) The row count

Why?

- r1) Any change to the content changes the hash, and identical content gives the same hash everywhere.
- r2) Hashes are shorter than file names.

Exercise 2. Predict the output (level 2)

What does this print?

```
1 import json
2
3 print(json.dumps({"min_base": 50, "cap": 5.0}, sort_keys=True))
```

Exercise 3. Fill the gap (level 2)

Hash a large file without loading it into memory.

```
1 with path.open("____(1)") as f:
2     digest = hashlib.____(2)(f, "sha256").hexdigest()
```

Exercise 4. Write the function (level 3)

Write two functions.

- `file_sha256(path: Path) -> str` returns the hex SHA-256 of the file's bytes.
- `config_sha256(config: dict[str, object]) -> str` returns the hex SHA-256 of `json.dumps(config, sort_keys=True)` encoded as UTF-8, so key order does not matter.

Starter code:

```
1 import hashlib
2 import json
3 from pathlib import Path
4
5
6 def file_sha256(path):
7     ...
8
```

```

9
10 def config_sha256(config):
11     return hashlib.sha256(str(config).encode()).hexdigest()

```

Exercise 5. Architecture design (level 4)

Choose what the run manifest records for a monthly client report built from a supplier CSV.

Options: SHA-256 of the input file, with the raw file kept unchanged, The input file name.

1. Input data
2. Settings
3. Code
4. Where the manifest lives

Sources

- `hashlib.file_digest` was added in Python 3.11. (verified: Python hashlib documentation; run on Python 3.11 in this repository.)

B6-L3 Pinned dependencies and lock files

Concept

`pyproject.toml` states what your project **accepts**: `pandas>=3.0,<4`. A **lock file** records what was **installed**: `pandas 3.0.6`, `numpy 2.4.6`, and every indirect dependency, each with a file hash. You need both.

- The ranges in `pyproject.toml` say which versions your code supports. Keep them honest: a lower bound you have tested, an upper bound at the next major version when the library breaks things across majors.
- The lock file makes every install identical: your laptop, a colleague's, CI and the scheduled job. Commit it.

With **uv** (examples use `uv 0.8`):

```

1 uv lock                                # resolve pyproject.toml ranges, write uv.lock
2 uv sync --locked                       # install exactly uv.lock; fail if it is out of
   date
3 uv lock --upgrade-package pandas      # move one package on purpose

```

`uv sync --locked` exits with an error when `pyproject.toml` has changed since the lock was written: "The lockfile at `uv.lock` needs to be updated, but `--locked` was provided." Use it in CI, so a dependency change without a new lock fails the build instead of resolving something new on the runner.

Without `uv`, `pip-compile --generate-hashes` (`pip-tools`) writes a fully pinned `requirements.txt` with hashes, and `pip install --require-hashes -r requirements.txt` refuses any file whose hash differs.

`pip freeze > requirements.txt` is a weaker substitute. It lists what happens to be in your environment, including packages you installed by hand for a one-off test, with no hashes and no record of why each is there.

Upgrade on purpose: one dependency at a time, on a branch, with tests. Record the reason in the commit message. Never let a fresh install pick up new versions silently.

Why it matters

A forecasting job ran on a fresh container every night. `requirements.txt` said `pandas>=2.0`. The night `pandas 3.0` was released, the container installed it. The job's text columns switched to the new `str` dtype, a `.str.contains` call behaved differently on missing values, and the job filtered out 6% of rows. Nobody had changed a line of code.

A committed lock file, installed with `uv sync --locked`, keeps the old `pandas` until someone upgrades it deliberately, with tests.

Worked example: weak code

```
1 # requirements.txt
❶ pandas
❷ scikit-learn>=1.0
4 numpy
❸ jupyter
```

1. **Line 2.** No bounds: any future major version installs on the next fresh build.
2. **Line 3.** A lower bound only. Every build resolves whatever is newest that day.
3. **Line 5.** A development tool mixed into the runtime requirements.

Worked example: strong code

```
1 # pyproject.toml
2 [project]
3 requires-python = ">=3.12"
❶ dependencies = ["pandas>=3.0,<4", "scikit-learn>=1.8,<2", "numpy>=2.4,<3"]
5
6 [dependency-groups]
❷ dev = ["pytest>=9", "ruff>=0.13", "jupyterlab>=4"]
8
9 # CI
❸ uv sync --locked --group dev
```

1. **Line 4.** Honest ranges: tested lower bounds, next major excluded.

- 2. **Line 7.** Development tools in their own group, not in the runtime install.
- 3. **Line 10.** Installs exactly uv.lock. A stale lock fails the build.

Ranges in pyproject.toml, exact versions and hashes in a committed lock file, and --locked in CI.

Recap card

- **pyproject.toml vs lock file** pyproject.toml: ranges you support. Lock: exact versions and hashes of everything installed.
- **uv sync --locked** Installs exactly uv.lock and fails if pyproject.toml changed without a new lock.
- **Hashes without uv** pip-compile --generate-hashes, then pip install --require-hashes -r requirements.txt.
- **Why not pip freeze?** It captures one environment's accidents, with no hashes and no reasons.

Practice

Exercise 1. Multiple choice (level 1)

What does a lock file add that `pandas>=3.0,<4` in pyproject.toml does not?

- a) The exact version of every direct and indirect dependency, with file hashes.
- b) A list of packages your code imports.
- c) Faster imports at runtime.
- d) Nothing. They hold the same information.

Why?

- r1) Every install then gets identical packages, on every machine.
- r2) It lets pip skip dependency resolution entirely on first install.

Exercise 2. Predict the output (level 2)

You add `polars>=1.30` to `dependencies` in pyproject.toml and push without running `uv lock`. CI runs `uv sync --locked`. What happens?

```
1 $ uv sync --locked
```

Exercise 3. Fill the gap (level 2)

Write a pinned requirements file with hashes using pip-tools, then install it so any mismatched file is refused.

```
1 pip-compile --____(1) requirements.in
2 python -m pip install --____(2) -r requirements.txt
```

Exercise 4. Put in order (level 3)

Order the steps to upgrade scikit-learn in a locked project.

- Commit pyproject.toml and uv.lock with the reason
- Create a branch
- `uv lock --upgrade-package scikit-learn`
- Raise the bound in pyproject.toml if the new version is outside the range
- Run the test suite and compare key outputs with the last release

Exercise 5. Architecture design (level 4)

Set up dependency management for a nightly scoring job.

Options: Tested lower bounds, next major excluded, in pyproject.toml, Package names only, no versions, Exact == pins in pyproject.toml, no lock.

1. Declared dependencies
2. Exact versions
3. Install in CI and the job

Sources

- `uv sync --locked` fails when uv.lock needs updating, with the message quoted in this lesson. (verified: Run with uv 0.8.17 in this repository's scratch directory.)
- `pip-compile --generate-hashes` and `pip install --require-hashes`. (search-only: pip-tools and pip documentation (not run here).)
- `Series.str.contains` returns `None` for a missing value on pandas 2.3 (object dtype) and `False` on pandas 3.0 (str dtype). (verified: Run on pandas 2.3.3 and 3.0.6 in this repository.)

Track B best-practice checklist

- ☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.
- ☐ ruff and mypy pass before every commit.
- ☐ Functions are pure where possible. Side effects sit at the edges.
- ☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.
- ☐ Inputs are validated with pandera or pydantic at every boundary.
- ☐ Seeds, config and data versions are recorded for every result.
- ☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.
- ☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.
- ☐ The metric matches the decision. Calibration and subgroup performance are checked.
- ☐ Every experiment is tracked in MLflow.
- ☐ Tests cover edge cases, including empty inputs and zeros.
- ☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B1-L1 One pyproject.toml and a src layout

B1-L1 exercise 1

Hints. (1) Which table never ships in the built package? (2) It was added by PEP 735. (3) `[dependency-groups]`.

Answer. b) `[dependency-groups]` Because: r1) Dependency groups (PEP 735) are named lists that are not part of the built distribution.

`[dependency-groups]` holds development-only requirements. `pip install --group dev (pip 25.1+)` installs them.

Common wrong answers.

- a: `[project] dependencies` install for every user of the package, in production too.
- c: That table configures how pytest runs. It does not install anything.
- d: A comment installs nothing and drifts from reality.
- b|r2: Right table, wrong reason. Groups can hold any development tools: tests, linting, docs, notebooks.

B1-L1 exercise 2

Hints. (1) The runtime key is plural. (2) The development table has a hyphenated name. (3) `dependencies` and `dependency-groups`.

Answer.

```
1 [project]
2 name = "churn"
3 requires-python = ">=3.11"
4 dependencies = ["pandas>=3.0,<3.1", "scikit-learn>=1.8,<1.10"]
5
6 [dependency-groups]
7 dev = ["pytest>=9", "ruff>=0.16", "mypy>=2.3"]
```

`[project].dependencies` lists what users need to run the package.

`[dependency-groups]` lists what developers need to work on it.

Common wrong answers.

- `g1 = requires`: The runtime list under `[project]` is called `dependencies`.

- `g2 = tool.dev: [tool.*]` tables configure tools. Development requirements go in `[dependency-groups]`.
- `g2 = optional-dependencies:` Optional dependencies (extras) ship with the package for users to opt into. Development-only tools belong in `[dependency-groups]`.

B1-L1 exercise 3

Hints. (1) `tomllib.loads(text)` returns a dict. (2) Use `.get` with defaults for the project table and the dependencies list. (3) Keep entries that contain none of `= < > ~ !`.

Answer.

```
1 import tomllib
2
3 CONSTRAINT_CHARS = set("=<>~!")
4
5
6 def unpinned(pyproject_text: str) -> list[str]:
7     """Runtime dependencies with no version constraint.
8
9     Args:
10         pyproject_text: Contents of pyproject.toml.
11
12     Returns:
13         Sorted requirement strings without =, <, >, ~ or !.
14     """
15     project = tomllib.loads(pyproject_text).get("project", {})
16     deps = project.get("dependencies", [])
17     return sorted(d for d in deps if not CONSTRAINT_CHARS & set(d))
```

`tomllib` ships with Python 3.11+. Checking characters is enough to spot a missing constraint, and `.get` handles projects that declare no dependencies.

B1-L1 exercise 4

Hints. (1) Why can Python not see `src/churn`? (2) What makes a package importable everywhere in the environment? (3) An editable install.

Answer. b) Run `pip install -e .` in the project's virtual environment.

Because: r1) The `src` layout is designed so that only the installed package is importable, which makes tests use it like any user would.

`pip install -e .` installs the package in editable mode, so imports resolve to `src/churn` and edits take effect without reinstalling.

Common wrong answers.

- a: Path hacks in every file defeat the point of the layout and break when files move.

- `c`: That removes the protection the layout gives: tests would import the working directory instead of the installed package.
- `d`: Copies drift. Tests must exercise the real package.
- `b|r2`: Right fix, wrong reason. `pytest` can import from folders. The `src` layout deliberately hides the package until it is installed.

B1-L1 exercise 5

Hints. (1) What must more than one program import? (2) What should CI run on every change? (3) What must never be cloned?

Answer.

- Shared logic in the package, imported by notebook and jobs.
- One-off exploration in notebooks, which import the package.
- Known-answer checks in tests/.
- Real data in governed storage, never in Git. Keep only small anonymised samples in the repository.

Package for shared logic, notebooks for exploration, tests for checks, governed storage for data.

Common wrong answers.

- `features = notebook`: The scoring job cannot import a notebook cell, so it would copy it, and the copies would drift.
- `extract = src`: Client data in the repository leaks with every clone. Keep it in governed storage.
- `known_answer = notebook`: A check in a notebook runs only when someone reruns the notebook. Put it in tests/ so CI runs it.

B1-L2 One virtual environment per project, and `python -m pip`

B1-L2 exercise 1

Hints. (1) How many Pythons might be on your PATH? (2) Which `pip` does `python -m pip` use? (3) The one belonging to that python.

Answer. b) It installs into the interpreter that `python` runs, whatever is first on PATH. Because: r1) `python -m pip` runs `pip` as a module of that exact interpreter.

`python -m pip` ties the install to the interpreter you will run. Bare `pip` is whichever `pip` script PATH finds first.

Common wrong answers.

- `a`: Speed is the same. The benefit is installing into the right interpreter.
- `c`: It installs into the current interpreter's environment, which inside a venv is the project only.
- `d`: Resolution is identical. Only the target interpreter is guaranteed.
- `b|r2`: Right answer, wrong reason. Bare pip still works. It may simply belong to a different Python.

B1-L2 exercise 2

Hints. (1) What does `sys.prefix` point to inside a venv? (2) And `sys.base_prefix`? (3) They are different folders.

Answer.

```
1 True
```

`sys.prefix` is the virtual environment, `sys.base_prefix` is the Python it was created from. They differ only inside a venv.

Common wrong answers.

- `False`: Inside a venv, `sys.prefix` points at the venv folder and `sys.base_prefix` at the base installation, so they differ.

B1-L2 exercise 3

Hints. (1) The standard library module that creates environments. (2) The flag for an editable install. (3) `venv` and `-e`.

Answer.

```
1 python -m venv .venv
2 source .venv/bin/activate
3 python -m pip install -e . --group dev
```

`python -m venv .venv` creates the environment. `pip install -e .` installs the project so edits under `src/` take effect immediately.

Common wrong answers.

- `g1 = virtualenv`: `virtualenv` is a separate package. The standard library module is `venv`.
- `g2 = -r: -r` reads a requirements file. Editable installs of the project use `-e .`.

B1-L2 exercise 4

Hints. (1) Inside a venv the two prefixes differ. (2) Raise when they are equal. (3) Read them inside the function body.

Answer.

```
1 import sys
2
3
4 def require_virtualenv() -> None:
5     """Stop when not running inside a virtual environment.
6
7     Raises:
8         RuntimeError: If sys.prefix equals sys.base_prefix.
9     """
10    if sys.prefix == sys.base_prefix:
11        raise RuntimeError("activate the project's virtual environment
    first")
```

Equal prefixes mean the system or base interpreter is running. Reading them at call time keeps the check correct in tests and in any process that changes them.

B1-L2 exercise 5

Hints. (1) Can one environment hold both constraints? (2) Where will the next developer look? (3) CI must match how each environment is used.

Answer.

- Two environments, because the constraint is real and tested.
- Two dependency groups in pyproject.toml, with the reason written next to the pin.
- Two CI jobs, so each environment is tested exactly as it is used.

When two tools conflict, give each its own pinned environment, record why in pyproject.toml, and test both in CI.

Common wrong answers.

- `envs = one_latest`: PySpark 4.0.4 fails with pandas 3.x: `assertDataFrameEqual` raises `ImportError`. One environment cannot satisfy both.
- `envs = one_old`: That works, but the analysis code then runs on an older pandas than intended. Separate environments keep each part current.
- `ci = one_job`: Installing both groups together recreates the conflict in CI.
- `declare = readme`: Chat messages are lost. Record the pin and its reason in pyproject.toml.

B1-L3 ruff and mypy in daily work

B1-L3 exercise 1

Hints. (1) How long is each list? (2) What does zip do when lengths differ? (3) It stops at the shortest.

Answer.

```
1 [(1, 2.0), (0, 1.0)]
```

zip stops at the shortest input, so the third respondent disappears. ruff's B905 asks for `strict=True`, which raises `ValueError` on unequal lengths (Python 3.10+).

Common wrong answers.

- `ValueError`: Only `zip(..., strict=True)` raises. Plain zip stops at the shorter input without a word.
- `[(1, 2.0), (0, 1.0), (1, None)]`: That is `itertools.zip_longest`. zip drops the extra element.

B1-L3 exercise 2

Hints. (1) When can share be None? (2) Is that case possible in real data? (3) Handle it.

Answer. b) Handle the None case before the multiplication. Because: r1) mypy found a real path where the value is None. Handling it removes the crash.

mypy found a reachable crash. Handle `None` explicitly, for example by returning `"n/a"`.

Common wrong answers.

- a: The ignore hides the warning and keeps the crash.
- c: The hint would then lie, and mypy would flag the `return None` instead. The crash stays.
- d: Weakening the checker hides this bug and every future one like it.
- b|r2: Right fix, wrong reason. This is a real crash on zero weight, not style.

B1-L3 exercise 3

Hints. (1) The flag that applies safe fixes. (2) The formatter subcommand. (3) The type checker's command.

Answer.

```

1 ruff check --fix .
2 ruff format .
3 mypy

```

`ruff check --fix .` applies safe fixes, `ruff format .` formats, and `mypy` type-checks with the settings in `pyproject.toml`.

Common wrong answers.

- `g1 = --unsafe-fixes`: Unsafe fixes can change behaviour. Use `--fix` for the safe ones, and review the rest by hand.
- `g3 = python -m pytest`: Tests matter, but the step after formatting in this routine is the type checker.

B1-L3 exercise 4

Hints. (1) Annotate: `flags: list[int], weights: list[float]`. (2) `zip(flags, weights, strict=True)`. (3) In report, store the share and return `'n/a'` when it is `None`.

Answer.

```

1 def weighted_share(flags: list[int], weights: list[float]) -> float |
    None:
2     """Weighted share of flagged rows, or None when the total weight is
    0."""
3     total = sum(weights)
4     if total == 0:
5         return None
6     return sum(w for f, w in zip(flags, weights, strict=True) if f ==
    1) / total
7
8
9 def report(flags: list[int], weights: list[float]) -> str:
10     """Share as a percentage string, or 'n/a' when undefined."""
11     share = weighted_share(flags, weights)
12     if share is None:
13         return "n/a"
14     return f"{share * 100:.1f}%"

```

The types make the `None` case visible, `strict=True` stops silent truncation, and `report` handles `None` on purpose.

B1-L3 exercise 5

Hints. (1) Prefer real type information. (2) When there is none, narrow the exception. (3) Never global.

Answer.

- Stubs exist: install them in the dev group, and `mypy` checks your use of the library.

- No stubs: silence only that module with a targeted override.
- Never switch it off globally, or you lose the check for libraries that do ship types.

Install stubs where they exist, override per module where they do not, and keep the global check on.

Common wrong answers.

- `stubs_exist = override`: Stubs exist, so installing them gives you real checks for free.
- `many = global`: A global setting hides missing stubs you could have installed, for every library, forever.

B2-L1 Pure functions, side effects at the edges

B2-L1 exercise 1

Hints. (1) Does the result depend on anything except the arguments? (2) Does the function change anything outside itself? (3) Only one answers no to both.

Answer. b) `def share(n, total): return n / total if total else None`
 Because: r1) Its result depends only on its arguments, and it changes nothing outside itself.

`share` depends only on `n` and `total` and changes nothing. The others mutate, read the clock, or write a file.

Common wrong answers.

- a: `append` mutates the caller's list, a side effect.
- c: It reads the clock, so the same input gives a different result each call.
- d: Writing a file is a side effect. It belongs at the edge.
- b|r2: Right function, wrong reason. Purity is about inputs and side effects, not length.

B2-L1 exercise 2

Hints. (1) Is `df` a copy or the caller's object? (2) Does column assignment change the object it is called on? (3) It does.

Answer.

```
1 ['x', 'flag']
```

`df` is the same object as `original`. Assigning a column changes it, so the caller sees `flag` appear. Use `df.assign(flag=...)` to return a new frame.

Common wrong answers.

- ['x']: Copy-on-Write protects against changes through derived objects, but `df["flag"] = ...` changes `df` itself, and `df` is the caller's object.

B2-L1 exercise 3

Hints. (1) Add a parameter for the year. (2) Use `df.assign(...)`, which returns a new frame. (3) `return df.assign(age=as_of_year - df["birth_year"])`.

Answer.

```
1 import pandas as pd
2
3
4 def add_age(df: pd.DataFrame, as_of_year: int) -> pd.DataFrame:
5     """Return a copy of df with age = as_of_year - birth_year."""
6     return df.assign(age=as_of_year - df["birth_year"])
```

The year is an input, so tests fix it. `assign` returns a new frame, so callers keep their data.

B2-L1 exercise 4

Hints. (1) When is the default list created? (2) Is it shared between calls? (3) Build a fresh list inside the function.

Answer. The bug is on line 1.

Fixed version:

```
1 def flag_low_bases(bases: dict[str, float], minimum: float) -> list[str]:
2     """Names of subgroups whose base is below minimum."""
3     return [name for name, base in bases.items() if base < minimum]
```

The default list is created once and shared by every call, a hidden state that makes the function impure. Build the list inside, here with a comprehension.

Common wrong answers.

- Line 5: `append` is where the state leaks, but the shared list is created on line 1.

B2-L1 exercise 5

Hints. (1) Does the step touch anything outside Python's memory? (2) Does its result depend only on its arguments? (3) I/O at the edges, logic in the core.

Answer.

- Reading and writing, logging and choosing the reference date: edge.

- Features and scoring, given all inputs as arguments: pure core.

A thin edge does I/O. A pure core does the thinking and gets the tests.

Common wrong answers.

- `read = core`: Reading storage depends on the world outside the function. Keep it at the edge.
- `write = core`: Writing and logging are side effects. Keep them at the edge.
- `features = edge`: Feature logic given its inputs is deterministic. Make it pure so it can be tested and reused.

B2-L2 Dataclasses as configuration objects

B2-L2 exercise 1

Hints. (1) What does `frozen=True` promise? (2) Is the failure silent or loud? (3) The exception is a subclass of `AttributeError`.

Answer. b) Raises `dataclasses.FrozenInstanceError`. Because: `r1` `frozen=True` makes the generated `setattr` raise on every field assignment. Frozen dataclasses raise `dataclasses.FrozenInstanceError` on assignment. Build a changed copy with `dataclasses.replace`.

Common wrong answers.

- a: That is the behaviour without `frozen=True`. Frozen instances reject assignment.
- c: Assignment never returns a copy. Use `dataclasses.replace(cfg, min_base=30)` for that.
- d: It fails loudly, which is the point: settings cannot drift during a run.
- b|r2: Right answer, wrong reason. Without `frozen=True` every field, int or not, accepts assignment.

B2-L2 exercise 2

Hints. (1) Does a dataclass check or convert types? (2) What does the generated `repr` show for a string? (3) It uses `repr()` of each value, so strings keep their quotes.

Answer.

```
1 Settings(min_base='30')
```

The hint `int` is for your editor and mypy. At runtime the dataclass stores `'30'`. Check or convert in `__post_init__`.

Common wrong answers.

- `Settings(min_base=30)`: Dataclasses do not convert values. The string '30' is stored as it is.
- `TypeError`: Type hints are not checked at runtime. The call succeeds and stores a string.

B2-L2 exercise 3

Hints. (1) A plain `= []` raises `ValueError` at class definition. (2) You need a function that builds a fresh list per instance. (3) The keyword is `default_factory`.

Answer.

```
1 from dataclasses import dataclass, field
2
3
4 @dataclass
5 class Run:
6     waves: list[str] = field(default_factory=list)
```

`field(default_factory=list)` calls `list()` for each new instance, so no two instances share one list.

Common wrong answers.

- `g1 = default: field(default=list)` stores the list class itself, not a new list.

B2-L2 exercise 4

Hints. (1) `ReportConfig(**raw)` passes each key as a keyword argument. (2) The generated `__init__` raises `TypeError` on an unknown or missing keyword. (3) Check `isinstance(self.min_base, int)` and `self.min_base > 0` in `__post_init__`.

Answer.

```
1 from dataclasses import dataclass
2
3
4 @dataclass(frozen=True)
5 class ReportConfig:
6     """Settings for one reporting run."""
7
8     weight_col: str
9     min_base: int = 50
10
11     def __post_init__(self) -> None:
12         if not isinstance(self.min_base, int) or self.min_base <= 0:
13             raise ValueError(f"min_base must be a positive int, got {self.min_base!r}")
```

```

14
15
16 def load_config(raw: dict[str, object]) -> ReportConfig:
17     """Build a ReportConfig from a parsed YAML or JSON mapping.
18
19     Args:
20         raw: Keys must match ReportConfig fields.
21
22     Returns:
23         A validated, frozen config.
24
25     Raises:
26         TypeError: On an unknown or missing key.
27         ValueError: On an invalid min_base.
28     """
29     return ReportConfig(**raw) # type: ignore[arg-type]

```

Unpacking the dict into the constructor lets the generated `__init__` reject unknown and missing keys. `__post_init__` catches bad values. `frozen=True` stops later edits.

B2-L2 exercise 5

Hints. (1) Which shape rejects a misspelt key? (2) When would you rather see a bad value: now or in hour three? (3) Globals make functions impure (lesson B2-L1).

Answer.

- A frozen dataclass: named fields, unknown keys rejected, no edits during the run.
- Check once at load time in **post_init**, so a bad file fails before any work starts.
- Pass the object as a parameter, so tests build their own config.

Build a frozen, validated object at the edge and pass it into the pure core.

Common wrong answers.

- `shape = dict`: A dict accepts any key, including typos, and any function can change it.
- `shape = globals`: Reading a file at import is a module-level side effect, and every test shares the same values.
- `when = use`: Checks spread across functions fail late, after hours of work, and drift apart.
- `pass = import`: A shared import is a global. Tests cannot give one function a different config.

B2-L3 When a class earns its place

B2-L3 exercise 1

Hints. (1) Which task has two phases? (2) What must survive between those phases? (3) Learned frequencies must be stored between fit and transform.

Answer. b) Learn category frequencies on training data and encode test data with them. Because: r1) State learned in one call must be reused in a later call.

Fit then transform needs stored state, the classic case for a class, as in scikit-learn's encoders.

Common wrong answers.

- a: One step with no state. A function taking the mapping is enough.
- c: One step with no state. A function is enough.
- d: Settings with no behaviour belong in a dataclass, not a full class.
- b|r2: Right answer, wrong reason. Size does not decide it. State across calls does.

B2-L3 exercise 2

Hints. (1) What is in self.seen after the first call? (2) Which of 2, 3 and 4 is not in it? (3) Only 4.

Answer.

```
1 [4]
```

The object keeps state between calls. Reuse it on a new wave and it drops rows the new wave needs.

Common wrong answers.

- [2, 3, 4]: self.seen still holds 1, 2 and 3 from the first call, so 2 and 3 are dropped.

B2-L3 exercise 3

Hints. (1) Move self.col into a parameter. (2) Compute the total once and test it for zero. (3) `df.assign(share=0.0)` when the total is zero.

Answer.

```
1 import pandas as pd
2
3
4 def add_share(df: pd.DataFrame, col: str) -> pd.DataFrame:
```

```

5     """Return df with share = col / column total, or 0.0 when the total is
      zero."""
6     total = df[col].sum()
7     if total == 0:
8         return df.assign(share=0.0)
9     return df.assign(share=df[col] / total)

```

The class held one value and did one thing. A function with two parameters is shorter, and the zero guard stops a NaN column.

B2-L3 exercise 4

Hints. (1) Which state should transform read? (2) Does transform change that state? (3) Refitting inside transform discards what fit learned.

Answer. The bug is on line 11.

Fixed version:

```

1 class Scaler:
2     """Standardise with the mean and sd learned in fit."""
3
4     def fit(self, values: list[float]) -> "Scaler":
5         n = len(values)
6         self.mean_ = sum(values) / n
7         self.sd_ = (sum((v - self.mean_) ** 2 for v in values) / n) ** 0.5
8         return self
9
10    def transform(self, values: list[float]) -> list[float]:
11        return [(v - self.mean_) / self.sd_ for v in values]

```

Refitting in `transform` throws away the learned state, the only reason the class exists. It also leaks test data into the scaling.

Common wrong answers.

- Line 6: The mean formula is correct. The problem is when it is recomputed.
- Line 12: The scaling formula is correct. It uses values that were just overwritten on line 11.

B2-L3 exercise 5

Hints. (1) Does it hold data, do something, or both? (2) Do several operations share the same data? (3) Delete the class: what breaks?

Answer.

- Recoding is one stateless step: a function.
- Settings with no behaviour: a frozen dataclass.

- A survey design shares the same data across several operations and must keep weights valid: a class.

Match the form to the need: function for a step, dataclass for settings, class for shared state with behaviour.

Common wrong answers.

- `recode = class`: One step, no state that outlives the call. A class adds construction with no benefit.
- `settings = class`: No behaviour to attach. A frozen dataclass gives fields, equality and immutability for free.
- `design = function`: Every function would need weights, strata and clusters passed again. A class keeps them together and valid.

B3-L1 Dtypes in pandas 3, nullable and string

B3-L1 exercise 1

Hints. (1) Can NumPy's int64 hold NaN? (2) What does pandas use for NaN by default? (3) A float.

Answer. c) float64 Because: r1) NumPy int64 cannot hold a missing value, so pandas stores NaN in a float column.

NaN is a float, so a missing value upcasts integers to float64. Ask for Int64 to keep integers.

Common wrong answers.

- a: int64 has no way to represent a missing value.
- b: Int64 is used only when you ask for it, for example with `dtype="Int64"` or `convert_dtypes()`.
- d: Numbers with None infer a numeric dtype, not object.
- c|r2: Right answer, wrong reason. `pd.Series([4, 2])` is int64. The None forces the float.

B3-L1 exercise 2

Hints. (1) What dtype does `read_csv` infer for 01234? (2) What does the blank row add? (3) A number with a missing value is float64.

Answer.

```
1 1234.0
```

`read_csv` reads 01234 as a number and the blank as NaN, so the column is float64 and the leading zero is gone. Pass `dtype={"postcode": "str"}`.

Common wrong answers.

- 01234: Without a dtype map, read_csv infers a number. The blank row makes it float64.
- 1234: The blank row forces float64, so the value prints with .0.

B3-L1 exercise 3

Hints. (1) IDs are labels, not quantities. (2) Which integer dtype holds pd.NA? (3) str and Int64.

Answer.

```
1 df = pd.read_csv(path, dtype={"respondent_id": "str", "household_size":  
    "Int64"})
```

str keeps IDs as written. Int64 is the nullable integer dtype.

Common wrong answers.

- g2 = int64: Lower-case int64 is NumPy's integer and fails on a blank value. Use capital-I Int64.
- g1 = int: IDs are labels. Reading them as numbers drops leading zeros.

B3-L1 exercise 4

Hints. (1) read_csv takes a dtype mapping. (2) IDs as str, counts as Int64. (3) pd.read_csv(io.StringIO(text), dtype=PANEL_DTYPES).

Answer.

```
1 import io  
2  
3 import pandas as pd  
4  
5 PANEL_DTYPES: dict[str, str] = {"respondent_id": "str", "household_size":  
    "Int64"}  
6  
7  
8 def read_panel(text: str) -> pd.DataFrame:  
9     """Read panel CSV text with explicit dtypes.  
10  
11     Args:  
12         text: CSV content with respondent_id and household_size.  
13  
14     Returns:  
15         A frame with str IDs and nullable integer counts.  
16     """  
17     return pd.read_csv(io.StringIO(text), dtype=PANEL_DTYPES)
```

The dtype map fixes both columns at load time. In a real pipeline it comes from config, not code.

B3-L1 exercise 5

Hints. (1) What does `mean()` do with `<NA>`? (2) What does it do with 0? (3) Filling blanks with 0 adds fake answers.

Answer. The bug is on line 7.

Fixed version:

```
1 import pandas as pd
2
3
4 def mean_household(df: pd.DataFrame) -> float:
5     """Mean household size among respondents who answered."""
6     sizes = df["household_size"].astype("Int64")
7     return float(sizes.mean())
```

`mean()` skips `<NA>` by default. Filling with 0 turns non-answers into households of size zero and pulls the mean down.

Common wrong answers.

- Line 6: `Int64` is right: it keeps blanks as `<NA>`, which `mean()` skips. The problem is what happens next.
- Line 8: `mean()` is fine on its own. It skips `<NA>`, but by then the blanks are zeros.

B3-L2 Vectorise instead of apply

B3-L2 exercise 1

Hints. (1) Look for the option with no per-row Python code. (2) A vectorised call takes whole columns as arguments. (3) `np.where(cond, a, b)` picks from `a` or `b` element by element in compiled code.

Answer. c) `np.where(df["reach_flag"] == 1, df["final_weight"], 0.0)`

Because: r1) It evaluates the condition and the choice on whole arrays in compiled code.

`np.where` takes whole arrays and runs one compiled loop. The other three options call Python code once per row.

Common wrong answers.

- `a: apply(axis=1)` calls the lambda once per row in Python. It measured about 500 times slower than `np.where` on one million rows.
- `b: A list comprehension` beats `apply`, but it still runs a Python loop over every value.
- `d: iterrows()` builds a Series for each row. It is the slowest option here.

- `c|r2`: Right choice, wrong reason. Length has nothing to do with speed. `np.where` works on whole arrays in compiled code.
- `c|r3`: Right choice, wrong reason. There is no `lambda`, and `np.where` runs on one core. It is fast because it avoids a Python call per row.

B3-L2 exercise 2

Hints. (1) `None` in a float Series becomes `NaN`. (2) What does any comparison with `NaN` return? (3) Every comparison with `NaN` is `False`.

Answer.

```
1 [False, True, False]
```

`None` becomes `NaN` in a float64 Series, and `NaN > 1` is `False`. So the missing value is treated as "not above 1". This is the missing-value trap from the concept.

Common wrong answers.

- `[False, True, None]`: A comparison on a float Series returns plain booleans. `NaN > 1` evaluates to `False`, not to a missing value.
- `[False, True, nan]`: Comparisons do not propagate `NaN` on float64 data. They return `False`, which is why missing values get silently mislabelled.
- `[True, True, False]`: `0.0 > 1` is `False`. Only 1.5 exceeds 1.

B3-L2 exercise 3

Hints. (1) The function takes a condition, a value for True and a value for False. (2) Its name is the SQL keyword for filtering rows. (3) `np.where(cond, value_if_true, value_if_false)`, with zero as the false value.

Answer.

```
1 df["weighted_reach"] = np.where(df["reach_flag"] == 1,
    df["final_weight"], 0.0)
```

`np.where(cond, df["final_weight"], 0.0)` keeps the weight where the flag is 1 and returns 0.0 elsewhere, as one float array.

Common wrong answers.

- `g1 = select`: `np.select` takes a list of conditions and a list of choices. For one condition and two outcomes, use `np.where`.
- `g1 = if`: There is no `np.if`. The vectorised if-else in NumPy is `np.where`.
- `g2 = None`: `None` turns the result into an object array. Use `0.0` so the column stays numeric.

B3-L2 exercise 4

Hints. (1) Compute the total weight first, and return early when it is zero. (2) `df['final_weight'].where(df['reach_flag'] == 1, 0.0)` keeps weights for reached rows only. (3) Sum the masked weights, divide by the total, and wrap the result in `float()`.

Answer.

```
1 import numpy as np
2 import pandas as pd
3
4
5 def weighted_reach_pct(df: pd.DataFrame) -> float | None:
6     """Weighted share of respondents with reach_flag == 1.
7
8     Args:
9         df: Rows with reach_flag and final_weight.
10
11     Returns:
12         The weighted share, or None when the total weight is zero.
13     """
14     total = df["final_weight"].sum()
15     if total == 0:
16         return None
17     reached = df["final_weight"].where(df["reach_flag"] == 1, 0.0).sum()
18     return float(reached / total)
```

Sum the weights once for the denominator and return `None` when it is zero, which covers the empty frame too. Mask weights by the flag, sum, divide and convert to `float`.

B3-L2 exercise 5

Hints. (1) `pd.cut` turns numbers into labelled bands and keeps missing values missing. (2) Bins `[-np.inf, 35, 55, np.inf]` with `right=False` give `[.., 35)`, `[35, 55)`, `[55, ..)`. (3) Wrap it in `df.assign(age_group=pd.cut(...))` so you return a new frame.

Answer.

```
1 import numpy as np
2 import pandas as pd
3
4 AGE_BINS = [-np.inf, 35, 55, np.inf]
5 AGE_LABELS = ["16-34", "35-54", "55+"]
6
7
8 def add_age_group(df: pd.DataFrame) -> pd.DataFrame:
9     """Return a copy of df with an age_group column. Missing ages stay
10         missing."""
11     return df.assign(
```

```

11     age_group=pd.cut(df["age"], bins=AGE_BINS, labels=AGE_LABELS,
12                       right=False)

```

`pd.cut` with left-closed bins (`right=False`) matches the original `<` comparisons exactly. It keeps `NaN` as missing, which fixes a bug the original had. `assign` returns a new frame.

B3-L2 exercise 6

Hints. (1) What is `NaN >= 60`? (2) `np.where` has only two outcomes, so a missing value gets one of them. (3) Build the band as a Series, then keep it only where `minutes.notna()`.

Answer. The bug is on line 4.

Fixed version:

```

1 def add_viewing_band(df: pd.DataFrame) -> pd.DataFrame:
2     """Band weekly minutes into heavy (60 or more) and light. Missing
3     minutes stay missing."""
4     minutes = df["minutes"]
5     band = pd.Series(np.where(minutes >= 60, "heavy", "light"),
6                      index=df.index)
7     return df.assign(viewing_band=band.where(minutes.notna()))

```

`NaN >= 60` is `False`, so `np.where` returns `"light"` for missing minutes. Wrap the result in a Series on the same index and apply `.where(minutes.notna())` to restore the missing values.

Common wrong answers.

- Line 3: Line 3 only selects the column. The missing value is lost where the condition is evaluated.
- Line 5: `assign` is right. It writes whatever `band` holds, and `band` is already wrong.

B3-L3 Method chaining and Copy-on-Write

B3-L3 exercise 1

Hints. (1) What does `df[mask]` return under Copy-on-Write? (2) Where does the second step write? (3) To a copy that is thrown away.

Answer. b) Nothing. pandas emits a `ChainedAssignmentError` warning. Because: r1) The first indexing step returns a new object, so the write lands on a temporary copy.

Two indexing steps mean two objects. The write hits the temporary one. Write in one step with `.loc`, or use `assign`.

Common wrong answers.

- `a`: Under Copy-on-Write the assignment goes to a temporary copy. Use `df.loc[mask, "weight"] = 0`.
- `c`: `ChainedAssignmentError` is a warning, not an exception. The code keeps running with `df` unchanged.
- `d`: The filter only selects rows. And the write never reaches `df` anyway.
- `b|r2`: Right answer, wrong reason. `df.loc[mask, col] = value` still changes `df`.

B3-L3 exercise 2

Hints. (1) Is `s` a view you can write through? (2) When does Copy-on-Write copy? (3) On the first write to shared data.

Answer.

```
1 [1, 2, 3]
```

`s` shares memory with `df` until you write to it. The write triggers a copy, so `df` keeps its values.

Common wrong answers.

- `[100, 2, 3]`: Under Copy-on-Write, `s` behaves as a separate object. Writing to it copies the data first, so `df` is unchanged.

B3-L3 exercise 3

Hints. (1) Which column names does each step need? (2) Which step depends on which rows are present? (3) Rename, filter, normalise, band.

Answer.

1. `.rename(columns={"wt": "weight"})`
2. `.loc[lambda d: d["age"] >= 18]`
3. `.assign(weight=lambda d: d["weight"] / d["weight"].mean())`
4. `.pipe(add_age_band, bins=BINS, labels=LABELS)`

Each step sees the output of the step above. Rename first so later steps use canonical names. A step that aggregates, such as a mean, must follow the filter that defines its population.

Common wrong answers.

- The normalising step reads the weight column, which does not exist until the rename.
- Normalising before the filter averages over minors too, so adult weights no longer average 1.0.

B3-L3 exercise 4

Hints. (1) Filter with `.loc[lambda d: ...]`. (2) Cap with `Series.clip(upper=cap)`. (3) Join them: `df.loc[...].assign(weight=lambda d: d["weight"].clip(upper=cap))`.

Answer.

```
1 import pandas as pd
2
3
4 def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
5     """Keep adults and cap their weights at cap."""
6     return df.loc[lambda d: d["age"] >= 18].assign(
7         weight=lambda d: d["weight"].clip(upper=cap)
8     )
```

`loc` then `assign` builds a new frame in two steps, with no temporary copies to write to.

B3-L3 exercise 5

Hints. (1) How many indexing steps does line 7 take before the `=`? (2) Where does the value land? (3) Use `.loc` in one step, or `Series.where` inside `assign`.

Answer. The bug is on line 7.

Fixed version:

```
1 import pandas as pd
2
3
4 def zero_invalid(df: pd.DataFrame) -> pd.DataFrame:
5     """Set weight to 0 where valid is False."""
6     return df.assign(weight=df["weight"].where(df["valid"], 0.0))
```

`out[mask]["weight"] = 0.0` writes to a temporary copy.

`df.assign(weight=df["weight"].where(df["valid"], 0.0))` keeps weights where valid and puts 0.0 elsewhere.

Common wrong answers.

- Line 6: `copy()` is harmless. Under Copy-on-Write it is also cheap until a write.
- Line 8: Returning `out` is fine. `out` never received the zeros.

B4-L1 Polars expressions and lazy frames

B4-L1 exercise 1

Hints. (1) `scan_` versus `read_`. (2) When does a `LazyFrame` run? (3) On `collect()`.

Answer. b) Nothing is read yet. It holds a query plan. Because: r1) A `LazyFrame` records steps and runs them only on `collect()`.

`scan_parquet` builds a plan. `collect()` optimises and runs it, pushing the filter to the reader.

Common wrong answers.

- a: That is what `pl.read_parquet` followed by filter does. `scan_parquet` is lazy.
- c: Close, but no rows are read until `collect()`. Then only w9 rows are read.
- d: Nothing is written. A `LazyFrame` is a plan in memory.
- b|r2: Right answer, wrong reason. Polars reads Parquet itself.

B4-L1 exercise 2

Hints. (1) Which values are in group w2? (2) What is the sum of no non-null values? (3) Polars returns the additive identity.

Answer.

```
1 [3.0, 0.0]
```

The sum of an empty set of values is 0 in Polars. A real zero and "no data" look the same, so report `null_count()` alongside totals.

Common wrong answers.

- `[3.0, None]`: Polars sums an all-null group to 0, not null. Check `null_count()` when that matters.
- `[3.0]`: Every group appears in the output, including w2.

B4-L1 exercise 3

Hints. (1) A window over groups that keeps every row. (2) The method that runs a `LazyFrame`. (3) `over` and `collect`.

Answer.

```
1 out = (  
2     lf.with_columns(share=pl.col("weight") /  
3         pl.col("weight").sum().over("wave"))  
4     .collect()  
5 )
```

`.over("wave")` broadcasts the wave total to each row. `collect()` runs the plan.

Common wrong answers.

- `g1 = group_by`: `group_by` collapses rows. `over` computes the group total and keeps every row.
- `g2 = execute`: A `LazyFrame` runs with `collect()`.

B4-L1 exercise 4

Hints. (1) Aggregate to wave and brand first. (2) Get the wave total with `.sum().over("wave")`. (3) Guard with `pl.when(total > 0).then(...).otherwise(None)`.

Answer.

```
1 import polars as pl
2
3
4 def brand_share(df: pl.DataFrame) -> pl.DataFrame:
5     """Weighted share of each brand within its wave. Grain: one row per
6     wave and brand.
7
8     Args:
9         df: Columns wave, brand, weight.
10
11     Returns:
12         Columns wave, brand, share. share is null when the wave total is
13         0.
14     """
15     return (
16         df.group_by("wave", "brand")
17         .agg(brand_weight=pl.col("weight").sum())
18         .with_columns(total=pl.col("brand_weight").sum().over("wave"))
19         .with_columns(
20             share=pl.when(pl.col("total") > 0)
21             .then(pl.col("brand_weight") / pl.col("total"))
22             .otherwise(None)
23         )
24         .select("wave", "brand", "share")
25         .sort("wave", "brand")
26     )
```

Aggregate to the output grain, broadcast the wave total with `over`, and guard the division so a zero total gives null instead of NaN.

B4-L1 exercise 5

Hints. (1) What does `pl.len()` count? (2) Does it count rows with a null score? (3) `mean()` skips nulls.

Answer. The bug is on line 8.

Fixed version:

```
1 import polars as pl
2
3
4 def mean_score(df: pl.DataFrame) -> pl.DataFrame:
5     """Mean score per region among respondents who answered."""
6     return (
7         df.group_by("region")
8         .agg(mean_score=pl.col("score").mean())
9         .sort("region")
10    )
```

`pl.len()` counts every row, including skipped answers, so the denominator is too big. `pl.col("score").mean()` divides by the non-null count.

Common wrong answers.

- Line 7: Grouping by region is right. The formula inside `agg` is the problem.
- Line 9: Sorting does not change values.

B4-L2 DuckDB, SQL over files and DataFrames

B4-L2 exercise 1

Hints. (1) Which options build a new SQL string from the value? (2) Which one keeps the SQL fixed? (3) The placeholder.

Answer. b) `con.execute("... WHERE region = ?", [region])` Because: r1) The value is sent as data, separate from the SQL text.

A ? placeholder keeps the SQL text fixed and passes the value as data.

Common wrong answers.

- a: An f-string pastes the value into the SQL. A quote in the value changes the query.
- c: Concatenation pastes the value into the SQL, the same risk as an f-string.
- d: % formatting is string pasting too, not a parameter.
- b|r2: Right answer, wrong reason. Safety comes from sending the value separately.

B4-L2 exercise 2

Hints. (1) How many rows does an aggregate without GROUP BY return? (2) What is SQL sum over no rows? (3) NULL, returned as None inside a tuple.

Answer.

```
1 (None,)
```

One row, holding NULL. Check `row[0]` is `None` before using the value.

Common wrong answers.

- `(0.0,)`: SQL sum over zero rows is NULL, which DuckDB returns as `None`.
- `None`: An aggregate without GROUP BY always returns one row. `fetchone()` gives a tuple holding `None`.

B4-L2 exercise 3

Hints. (1) The DuckDB table function for Parquet files. (2) The positional placeholder. (3) `read_parquet` and `?`.

Answer.

```
1 con.execute(  
2     "SELECT count(*) FROM read_parquet('extracts/*.parquet') WHERE wave  
3     = ?",  
4     [wave],  
5 ).fetchone()
```

`read_parquet` with a glob reads all files as one table. `?` binds the first value in the list.

Common wrong answers.

- `g2 = '{wave}'`: That is f-string pasting, and the string is not even an f-string here. Use a `?` placeholder.
- `g1 = read_csv`: The files are Parquet. `read_csv` would fail to parse them.

B4-L2 exercise 4

Hints. (1) Replace the f-string with a `?` placeholder and a list of values. (2) What does `sum` return when no rows match? (3) Return `0.0` when `row[0]` is `None`, else `float(row[0])`.

Answer.

```
1 import duckdb  
2  
3 REGION_TOTAL_SQL = "SELECT sum(weight) FROM responses WHERE region = ?"  
4  
5  
6 def region_total(con: duckdb.DuckDBPyConnection, region: str) -> float:  
7     """Total weight for one region.  
8  
9     Args:  
10         con: Connection holding a responses table.  
11         region: Region name, passed as a query parameter.
```

```

12
13     Returns:
14         The weight total, or 0.0 when no rows match.
15     """
16     row = con.execute(REGION_TOTAL_SQL, [region]).fetchone()
17     if row is None or row[0] is None:
18         return 0.0
19     return float(row[0])

```

The placeholder keeps the SQL fixed, so quotes in the value are just characters. The None check turns SQL's NULL into the 0.0 the caller expects.

B4-L2 exercise 5

Hints. (1) Can a placeholder stand for a table name? (2) What stops arbitrary text becoming SQL? (3) Globals make testing hard (lesson B2-L1).

Answer.

- Values such as the wave go in as parameters.
- Identifiers such as table names cannot be parameters. Check them against an allowlist before building the SQL.
- Pass one connection in, so tests supply an in-memory database.

Parameters for values, an allowlist for identifiers, and a connection passed in.

Common wrong answers.

- `wave = fstring`: A quote in the wave changes the SQL. Use a parameter.
- `table = param`: Placeholders bind values, not identifiers. `FROM ?` is not valid SQL for a table name.
- `table = raw`: Any text the user types becomes SQL. Check it against the allowed tables first.
- `con = default`: The default connection is a shared global. Tests cannot swap in their own.

B4-L3 Choosing pandas, Polars, DuckDB or Spark

B4-L3 exercise 1

Hints. (1) Can the whole dataset sit in RAM? (2) Which tools aggregate while reading? (3) Scan-based engines on one machine.

Answer. b) DuckDB or Polars lazy, aggregating while reading Because: r1) Both read Parquet in chunks and push filters and aggregation into the scan, so the full data never sits in memory.

DuckDB and Polars lazy read in chunks and aggregate as they go. The result is small.

Common wrong answers.

- **a**: pandas loads the whole selection into memory first. 40 GB does not fit in 16 GB.
- **c**: A cluster works, but a one-off aggregate on one machine does not need one.
- **d**: Manual chunking re-implements what DuckDB and Polars do for you, with more room for mistakes.
- **b|r2**: Right answer, wrong reason. Age is not the point. Out-of-memory scanning is.

B4-L3 exercise 2

Hints. (1) Where is `respondent_id` after `set_index`? (2) Does Polars have an index? (3) The default is `include_index=False`.

Answer.

```
1 ['score']
```

The ID lives in the pandas index, and `from_pandas` drops the index by default. Call `reset_index()` first.

Common wrong answers.

- `['respondent_id', 'score']`: Polars has no index. `from_pandas` drops it unless you pass `include_index=True`.

B4-L3 exercise 3

Hints. (1) DuckDB relations have `.df()`, `.pl()` and `.fetchall()`. (2) Polars converts to pandas with a `to_` method. (3) `pl` and `to_pandas`.

Answer.

```
1 top_brands = con.sql("SELECT brand, sum(weight) AS total FROM responses
    GROUP BY brand").pl()
2 for_model = top_brands.to_pandas()
```

`.pl()` returns a Polars DataFrame from DuckDB. `.to_pandas()` converts Polars to pandas through Arrow.

Common wrong answers.

- `g1 = df`: `.df()` returns pandas. The first step asks for Polars.
- `g2 = df`: Polars DataFrames convert with `to_pandas()`.

B4-L3 exercise 4

Hints. (1) How much data exists after line 7? (2) Where should the filter run? (3) Scan, filter, aggregate, collect.

Answer. The bug is on line 7.

Fixed version:

```
1 import polars as pl
2
3
4 def wave_mean(path: str, wave: str) -> float:
5     """Mean weight for one wave."""
6     result = (
7         pl.scan_parquet(path)
8         .filter(pl.col("wave") == wave)
9         .select(pl.col("weight").mean())
10        .collect()
11        .item()
12    )
13    return float(result)
```

Line 7 reads every wave eagerly, then copies it into pandas. A lazy scan pushes the wave filter into the reader and returns one number.

Common wrong answers.

- Line 8: The filter is correct, but by then every wave is already in memory twice.

B4-L3 exercise 5

Hints. (1) Where does the data live? (2) How big is it next to memory? (3) Which library does the next step need?

Answer.

- Small data feeding the Python modelling stack: pandas.
- SQL across many local files: DuckDB.
- Data already in the lakehouse, scheduled: Spark.
- Large local files with a transform pipeline: Polars lazy.

Size against memory, location of the data, and the next library in line decide the engine.

Common wrong answers.

- model = spark: 50,000 rows fit in memory many times over. Spark adds start-up time and a cluster for no gain.
- nightly = pandas: Pulling lakehouse tables into one pandas process moves the data away from the compute that already sits next to it.

- transform = `pandas`: pandas would load most of 25 GB into 32 GB of RAM. A lazy scan keeps memory in check.

B5-L1 DataFrame schemas with pandera

B5-L1 exercise 1

Hints. (1) Without lazy, when does validation stop? (2) What does the exception hold with lazy? (3) `failure_cases`, one row per failure.

Answer. b) Every failure is collected and raised together as `SchemaErrors`. Because: r1) You see all problems in one run, with `failure_cases` listing each one.

Without `lazy`, the first failing check raises `SchemaError`. With it, pandera runs every check and raises `SchemaErrors` with all failures.

Common wrong answers.

- a: Validation runs at once. lazy refers to collecting errors, not delaying them.
- c: pandera raises. Dropping rows needs an explicit choice, not a flag on `validate`.
- d: All checks run. lazy changes how failures are reported.
- b|r2: Right answer, wrong reason. lazy runs every check, so it is not faster.

B5-L1 exercise 2

Hints. (1) Which values fail `gt(0)`? (2) Does lazy stop at the first? (3) All failing values are listed.

Answer.

```
1 [-2.0, -3.0]
```

Both negative weights fail. `lazy=True` lists both in `failure_cases`.

Common wrong answers.

- `[-2.0]`: With `lazy=True` pandera reports every failing value, not only the first.
- `[1, 2]`: `failure_case` holds the failing values. The row positions are in the `index` column.

B5-L1 exercise 3

Hints. (1) A Column argument about repeated values. (2) A schema argument about extra columns. (3) `unique` and `strict`.

Answer.

```
1 schema = pa.DataFrameSchema(  
2     {"respondent_id": pa.Column(str, unique=True), "score":  
3     pa.Column(int)},  
4     strict=True,  
5 )
```

`unique=True` enforces the grain. `strict=True` fails on any unexpected column.

Common wrong answers.

- `g2 = coerce`: `coerce` converts dtypes. `strict` rejects unknown columns.
- `g1 = nullable`: `nullable` allows nulls. `unique` rejects duplicates.

B5-L1 exercise 4

Hints. (1) Each column is `pa.Column(dtype, checks..., options)`. (2) Columns reject nulls unless `nullable=True`. (3) `strict` and `coerce` go on the `DataFrameSchema`, `lazy` on `validate`.

Answer.

```
1 import pandas as pd  
2 import pandera.pandas as pa  
3  
4 SCORES = pa.DataFrameSchema(  
5     {  
6         "respondent_id": pa.Column(str, unique=True),  
7         "nps": pa.Column(int, pa.Check.in_range(0, 10)),  
8         "comment": pa.Column(str, nullable=True),  
9     },  
10    strict=True,  
11    coerce=True,  
12 )  
13  
14  
15 def validate_scores(df: pd.DataFrame) -> pd.DataFrame:  
16     """Validate an NPS extract. Grain: one row per respondent.  
17  
18     Raises:  
19         pa.errors.SchemaErrors: With every failing check.  
20     """  
21     return SCORES.validate(df, lazy=True)
```

The schema names every column, enforces the grain with `unique`, bounds the score, and allows nulls only where they are expected.

B5-L1 exercise 5

Hints. (1) What does `lazy=True` raise? (2) Is `SchemaErrors` a subclass of `SchemaError`? (3) Even if it were, what does the `except` branch return?

Answer. The bug is on line 12, 13.

Fixed version:

```
1 import pandas as pd
2 import pandera.pandas as pa
3
4 WEIGHTS = pa.DataFrameSchema({"weight": pa.Column(float,
5     pa.Check.gt(0))}, strict=True)
6
7 def load_weights(path: str) -> pd.DataFrame:
8     """Read and validate weights.
9
10    Raises:
11        pa.errors.SchemaErrors: When any check fails.
12    """
13     return WEIGHTS.validate(pd.read_csv(path), lazy=True)
```

Two faults. `lazy=True` raises `SchemaErrors`, which `except SchemaError` does not catch, so the handler is dead code for value failures. And when it does catch, it returns the unvalidated frame. Let the error raise, so the run stops.

Common wrong answers.

- Line 4: The schema is sound: positive, non-null weights, no extra columns.
- Line 11: Validating with `lazy=True` is right. What happens when it fails?

B5-L2 pydantic models for config and input records

B5-L2 exercise 1

Hints. (1) What is the default value of the extra setting? (2) Does `ignore` warn? (3) Change it with `model_config`.

Answer. b) The key is ignored silently. Because: r1) The default extra setting is `"ignore"`. Set `extra="forbid"` to reject unknown keys.

Default `extra="ignore"` drops unknown keys. Config models should use `extra="forbid"`.

Common wrong answers.

- a: Only with `ConfigDict(extra="forbid")`. The default ignores extras.
- c: pydantic does not guess spellings. `minbase` is a different key.
- d: No warning. That is why `extra="forbid"` matters for config files.

- `b|r2`: Right answer, wrong reason. It checks keys when you set `extra="forbid"`.

B5-L2 exercise 2

Hints. (1) Is "30" accepted in lax mode? (2) Does `model_copy` run validators? (3) It does not.

Answer.

```
1 -1
```

Lax mode converts "30" to 30. `model_copy(update=...)` then sets -1 without validation. Rebuild with `model_validate` instead.

Common wrong answers.

- 30: `model_copy` applies the update. It just does not validate it.
- `ValidationError: model_copy(update=...)` skips validation, so `gt=0` is never checked.

B5-L2 exercise 3

Hints. (1) The extra setting that raises. (2) The config key that blocks assignment. (3) Less than or equal.

Answer.

```
1 class Response(BaseModel):
2     model_config = ConfigDict(extra="forbid", frozen=True)
3
4     respondent_id: str
5     nps: int = Field(ge=0, le=10)
```

`extra="forbid"`, `frozen=True` and `le=10` encode the rules in the model.

Common wrong answers.

- `g1 = ignore`: `ignore` is the default and drops unknown keys silently.
- `g3 = lt`: `lt=10` excludes 10. NPS allows 10, so use `le`.

B5-L2 exercise 4

Hints. (1) Bounds go in `Field(gt=...)`. (2) Unknown keys and assignment are controlled in `model_config`. (3) `ConfigDict(extra="forbid", frozen=True)` and `model_validate(raw)`.

Answer.

```
1 from pydantic import BaseModel, ConfigDict, Field
2
3
4 class WeightingConfig(BaseModel):
5     """Settings for one weighting run."""
```

```

6
7     model_config = ConfigDict(extra="forbid", frozen=True)
8
9     weight_col: str
10    min_base: int = Field(default=50, gt=0)
11    cap: float = Field(default=5.0, gt=1.0)
12
13
14    def load_config(raw: dict[str, object]) -> WeightingConfig:
15        """Validate a parsed YAML or JSON mapping.
16
17        Raises:
18            pydantic.ValidationError: On any invalid, missing or unknown key.
19        """
20        return WeightingConfig.model_validate(raw)

```

Every rule lives in the model: required fields, bounds, no extras, no changes. `model_validate` reports all failures at once.

B5-L2 exercise 5

Hints. (1) Which pydantic methods validate? (2) Does `model_copy`? (3) Rebuild with `model_validate` from `model_dump()`.

Answer. The bug is on line 14.

Fixed version:

```

1  from pydantic import BaseModel, ConfigDict, Field
2
3
4  class Cfg(BaseModel):
5      """Run settings."""
6
7      model_config = ConfigDict(extra="forbid", frozen=True)
8
9      min_base: int = Field(default=50, gt=0)
10
11
12  def with_min_base(cfg: Cfg, value: int) -> Cfg:
13      """Return a validated copy of cfg with a new min_base."""
14      return Cfg.model_validate(**cfg.model_dump(), "min_base": value)

```

`model_copy(update=...)` skips validation. Dump, merge and `model_validate` so the bound applies to the new value.

Common wrong answers.

- Line 9: The bound is right. It is never checked on the copy.
- Line 7: `frozen=True` is right. `model_copy` builds a new object, so `frozen` does not stop it.

B5-L3 Validate at the boundaries, with severity

B5-L3 exercise 1

Hints. (1) Where does data you do not control arrive? (2) Where do you make promises to someone else? (3) In and out.

Answer. b) After reading, and before writing. Because: r1) Those are the boundaries where data changes owner. Tests cover the pure middle.

Validate what you receive and what you deliver. Test the logic in between.

Common wrong answers.

- a: Checks inside a tested pure core add noise. The core's inputs are already validated.
- c: A bad input would pass through every transform first, and the error would name the wrong step.
- d: Unit tests check your code on test data. They say nothing about this week's supplier file.
- b|r2: Right answer, wrong reason. The point is ownership boundaries, not speed.

B5-L3 exercise 2

Hints. (1) Is 0.0 truthy? (2) Which branch runs? (3) The fallback.

Answer.

```
1 no data
```

0.0 is falsy, so the best possible result is reported as missing data. Test with `is not None`.

Common wrong answers.

- `pass`: 0.0 is falsy, so `if share:` skips the comparison. Use `if share is not None:`.

B5-L3 exercise 3

Hints. (1) Check input before using it. (2) Check output before delivering it. (3) Log before raising.

Answer.

1. Read the supplier file
2. Run input checks and collect results
3. Log all results, then raise if any CRITICAL failed
4. Run the pure transforms

5. Run output checks on the result

6. Write the client table

Read, check, enforce, transform, check, write. Each boundary gets its checks before the data moves on.

Common wrong answers.

- Input checks must run before the transforms, so bad input never reaches them.
- Output checks after writing are too late. The client already has the table.

B5-L3 exercise 4

Hints. (1) Compute n once. The share is None when n is 0. (2) Build one QaResult per check. Use `share is not None` and `share <= max_missing`. (3) enforce filters on `severity == "CRITICAL"` and not passed.

Answer.

```
1 from dataclasses import dataclass
2 from typing import Literal
3
4 import pandas as pd
5
6 Severity = Literal["CRITICAL", "WARN"]
7
8
9 @dataclass(frozen=True)
10 class QaResult:
11     """One check outcome."""
12
13     check: str
14     severity: Severity
15     passed: bool
16     observed: float | None
17
18
19 def run_checks(df: pd.DataFrame, min_rows: int, max_missing: float) ->
20     list[QaResult]:
21     """Evaluate input checks. Never raises.
22
23     Args:
24         df: Input with a weight column.
25         min_rows: Minimum acceptable row count.
26         max_missing: Largest acceptable share of null weights.
27
28     Returns:
29         One QaResult per check.
30     """
31     n = len(df)
32     share = None if n == 0 else float(df["weight"].isna().sum()) / n
```

```

32     return [
33         QaResult("min_rows", "CRITICAL", n >= min_rows, float(n)),
34         QaResult("missing_weight", "WARN", share is not None and share <=
max_missing, share),
35     ]
36
37
38 def enforce(results: list[QaResult]) -> None:
39     """Stop the run when any CRITICAL check failed.
40
41     Raises:
42         RuntimeError: Naming each failed CRITICAL check.
43     """
44     failed = [r.check for r in results if r.severity == "CRITICAL" and
not r.passed]
45     if failed:
46         raise RuntimeError(f"critical checks failed: {failed}")

```

`run_checks` measures and never raises, so every result reaches the log.

`enforce` decides, and only CRITICAL failures stop the run.

B5-L3 exercise 5

Hints. (1) Does the failure make every output wrong? (2) Or does it affect one cell, handled by a known rule? (3) Grain and weights are foundations.

Answer.

- Duplicate respondents and broken weights make every figure wrong: CRITICAL.
- A small base is a reporting flag, handled by suppression: WARN.
- A rise in missing age is worth a look but does not invalidate the run: WARN.

CRITICAL for failures that invalidate the whole output. WARN for local issues with a known handling rule.

Common wrong answers.

- `grain = warn`: Duplicates double-count respondents in every total. Nothing downstream is correct.
- `weights_sum = warn`: If weights do not sum to the sample, every weighted figure is off. Stop the run.
- `base = critical`: Small bases are expected in some cells. Flag and suppress them. Do not stop the whole run.

B6-L1 Seeds and random generators

B6-L1 exercise 1

Hints. (1) Which option has no shared global state? (2) Which one can a test control? (3) `default_rng`.

Answer. b) `rng = np.random.default_rng(42)`, passed to the functions that need it Because: r1) Each Generator owns its state, so no other code can change the stream it gives your function.

`default_rng(seed)` gives an independent, seeded generator. Pass it in so each function's randomness is explicit.

Common wrong answers.

- a: This seeds a global generator that every library shares. Any extra call anywhere shifts your numbers.
- c: The standard library seed does not affect NumPy's generators.
- d: Unseeded runs cannot be reproduced or debugged.
- b|r2: Right answer, wrong reason. `np.random.seed` still exists as legacy API. Its global state is the problem.

B6-L1 exercise 2

Hints. (1) Does the generator's state change after a draw? (2) Is the second draw from the same position? (3) It continues the stream.

Answer.

```
1 False
```

The generator advances. Two draws differ, and rerunning the script repeats both.

Common wrong answers.

- True: One generator moves on after each draw. Only a fresh generator with the same seed repeats the first draw.

B6-L1 exercise 3

Hints. (1) The function that returns a Generator. (2) The Generator method that makes child generators. (3) `default_rng` and `spawn`.

Answer.

```
1 rng = np.random.default_rng(20260925)
2 worker_rngs = rng.spawn(4)
```

`default_rng` builds the parent. `spawn(4)` returns four independent children.

Common wrong answers.

- `g1 = seed: np.random.seed` sets the global generator and returns `None`.

B6-L1 exercise 4

Hints. (1) `hashlib.sha256(data)` needs bytes:

`respondent_id.encode("utf-8")`. (2) `.hexdigest()` returns hex text. `int(text, 16)` reads it. (3) Finish with `% n_buckets`.

Answer.

```
1 import hashlib
2
3
4 def bucket(respondent_id: str, n_buckets: int) -> int:
5     """Stable bucket for an ID, the same in every process.
6
7     Args:
8         respondent_id: The ID to bucket.
9         n_buckets: Number of buckets.
10
11     Returns:
12         An integer from 0 to n_buckets - 1.
13     """
14     digest = hashlib.sha256(respondent_id.encode("utf-8")).hexdigest()
15     return int(digest, 16) % n_buckets
```

SHA-256 of the ID's bytes is the same everywhere, so the bucket is too.

`hash()` is salted per process for strings.

B6-L1 exercise 5

Hints. (1) Which generator does `np.random.choice` use? (2) Who else can move that generator? (3) Use a local `default_rng(seed)`.

Answer. The bug is on line 6, 9.

Fixed version:

```
1 import numpy as np
2
3
4 def bootstrap_means(values: list[float], n_boot: int, seed: int) ->
5     list[float]:
6     """Bootstrap means of values, reproducible from seed."""
7     rng = np.random.default_rng(seed)
8     data = np.asarray(values)
9     return [float(rng.choice(data, size=len(data), replace=True).mean())
10             for _ in range(n_boot)]
```

`np.random.seed` and `np.random.choice` use one global generator that any imported code advances. A local `default_rng(seed)` belongs to this function alone.

Common wrong answers.

- Line 10: Taking the mean is fine. The draws come from a shared global generator.

B6-L2 Record the data, config and code behind every result

B6-L2 exercise 1

Hints. (1) Which option changes when one value in the file changes? (2) Which stays the same when the file is copied? (3) A content hash.

Answer. c) The SHA-256 of the file's bytes Because: r1) Any change to the content changes the hash, and identical content gives the same hash everywhere.

A content hash is tied to the bytes, not to the name or the file system.

Common wrong answers.

- a: Suppliers resend changed files under the same name.
- b: Copying a file changes its modified time without changing content, and edits can keep it.
- d: A file with changed values and the same number of rows has the same count.
- c|r2: Right answer, wrong reason. Length does not matter. Content identity does.

B6-L2 exercise 2

Hints. (1) What does `sort_keys` do? (2) Which comes first alphabetically? (3) c before m.

Answer.

```
1 {"cap": 5.0, "min_base": 50}
```

Sorted keys make the text, and so the hash, independent of the order you wrote the config in.

Common wrong answers.

- `{"min_base": 50, "cap": 5.0}`: `sort_keys=True` orders keys alphabetically: cap before min_base.

B6-L2 exercise 3

Hints. (1) Hashes work on bytes. (2) Python 3.11 added a hashlib helper for file objects. (3) `rb` and `file_digest`.

Answer.

```
1 with path.open("rb") as f:
2     digest = hashlib.file_digest(f, "sha256").hexdigest()
```

Binary mode gives the exact bytes. `hashlib.file_digest` reads them in chunks.

Common wrong answers.

- `g1 = r`: Text mode decodes and may change line endings. Hash the raw bytes with `rb`.

B6-L2 exercise 4

Hints. (1) Open the file in binary mode. (2) `hashlib.file_digest(f, "sha256")` or `hashlib.sha256(path.read_bytes())`. (3) For the config, `json.dumps(config, sort_keys=True).encode("utf-8")`.

Answer.

```
1 import hashlib
2 import json
3 from pathlib import Path
4
5
6 def file_sha256(path: Path) -> str:
7     """SHA-256 of a file's bytes, read in chunks.
8
9     Args:
10         path: File to hash.
11
12     Returns:
13         The hex digest.
14     """
15     with path.open("rb") as f:
16         return hashlib.file_digest(f, "sha256").hexdigest()
17
18
19 def config_sha256(config: dict[str, object]) -> str:
20     """SHA-256 of a config, independent of key order.
21
22     Args:
23         config: A JSON-serialisable mapping.
24
25     Returns:
26         The hex digest of the sorted JSON text.
27     """
28     text = json.dumps(config, sort_keys=True)
```

Hash content, not names, and serialise configs in a fixed key order before hashing.

B6-L2 exercise 5

Hints. (1) What changes without changing its name? (2) What would you compare between two runs? (3) Store it where the next analyst will look.

Answer.

- Content hash of an immutable input.
- Hash of the loaded config, so defaults and key order are handled.
- Git commit plus a dirty flag.
- Stored with the output and in the run log, where comparisons are easy.

Record identities, not names: content hashes and commits. Keep them where a diff between runs is one query away.

Common wrong answers.

- `data = name`: A resent file keeps its name. Only a content hash shows the data changed.
- `config = file_path`: The file at that path changes later, and defaults are not in it.
- `code = notebook`: The notebook changes under the same name. The commit pins the code.
- `store = chat`: Chat is hard to search and compare. Keep manifests with the output.

B6-L3 Pinned dependencies and lock files

B6-L3 exercise 1

Hints. (1) How many versions does a range accept? (2) What about packages you did not list, such as numpy under pandas? (3) Exact versions plus hashes.

Answer. a) The exact version of every direct and indirect dependency, with file hashes. Because: r1) Every install then gets identical packages, on every machine.

Ranges declare what you support. The lock pins what you run, down to every indirect dependency.

Common wrong answers.

- `b`: Neither file is generated from imports. The lock records what the resolver chose.
- `c`: Locks affect installation, not import speed.
- `d`: A range accepts many versions. The lock records exactly one of each, including indirect dependencies.
- `a|r2`: Right answer, wrong reason. The benefit is identical installs, not skipped work.

B6-L3 exercise 2

Hints. (1) Does `uv.lock` include polars? (2) What does `--locked` assert? (3) That the lock would stay unchanged.

Answer.

```
1 The lockfile at `uv.lock` needs to be updated, but `--locked` was
   provided. To update the lockfile, run `uv lock`.
```

`--locked` asserts the lock matches `pyproject.toml`. It does not, so the command exits with an error and tells you to run `uv lock`.

Common wrong answers.

- `Installed polars`: `--locked` forbids changing the lock. The build fails instead of resolving on the runner.

B6-L3 exercise 3

Hints. (1) `pip-compile` has a flag that adds sha256 lines. (2) `pip install` has a flag that enforces them. (3) `generate-hashes` and `require-hashes`.

Answer.

```
1 pip-compile --generate-hashes requirements.in
2 python -m pip install --require-hashes -r requirements.txt
```

Hashes turn a pin into a guarantee: a different file with the same version number is rejected.

Common wrong answers.

- `g1 = upgrade`: `--upgrade` moves pins to newer versions. The task asks for hashes.

B6-L3 exercise 4

Hints. (1) The lock respects the range in `pyproject.toml`. (2) Tests come before the commit. (3) Branch, bound, lock, test, commit.

Answer.

1. Create a branch
2. Raise the bound in `pyproject.toml` if the new version is outside the range
3. `uv lock --upgrade-package scikit-learn`
4. Run the test suite and compare key outputs with the last release
5. Commit `pyproject.toml` and `uv.lock` with the reason

Widen the range, re-lock one package, prove it with tests, then commit both files together.

Common wrong answers.

- Locking before raising the bound resolves within the old range, so the upgrade you want is not allowed yet.
- Test before you commit, so a broken upgrade never lands.

B6-L3 exercise 5

Hints. (1) Which file says what you support, and which says what you run? (2) Where do indirect dependencies get pinned? (3) What makes CI fail on a stale lock?

Answer.

- Ranges in `pyproject.toml` say what the code supports.
- The committed lock pins every package with hashes.
- `uv sync --locked` installs exactly that, and fails on a stale lock.

Ranges for intent, a lock for fact, and a locked install everywhere the code runs.

Common wrong answers.

- `declare = none`: No bounds means any future major version is acceptable, which is rarely true.
- `declare = exact`: Exact pins in `pyproject.toml` miss indirect dependencies and block security fixes. Pin exactly in the lock.
- `lock = freeze`: freeze captures whatever is on one laptop, one-off installs included, with no hashes.
- `install = fresh`: Fresh resolution picks up new releases silently. That is how a job breaks with no code change.